Machine Learning (2026) 115:237
https://doi.org/10.1007/s10994-026-07045-7

Large Language Model Guided Series Saliency Module for Accurate and Interpretable Multivariate Time Series Forecasting

Qingyi Pan[1,2] · **Liyuan Wang**[2] · **Xiaoyu Chen**[2] · **Ning Chen**[2]

Received: 7 March 2025 / Revised: 22 January 2026 / Accepted: 1 April 2026
© The Author(s), under exclusive licence to Springer Science+Business Media LLC, part of Springer Nature 2026

Abstract
Time series data analysis is an essential task in numerous critical applications, such as air quality assessment and stock price prediction, where the goal is to extract underlying patterns and predict future values. Despite the recent strong performance of deep models in multivariate time series forecasting, their failure to utilize essential trend information may hinder their effectiveness in filtering out anomalous patterns such as sawtooth shapes and spikes, thereby negatively affecting both accuracy and interpretability. To address these challenges, we propose a Series Saliency (SS) module that effectively perceives key trend information during both training and interpretation processes while exploring the previously uncharted sample space. In the training process, we leverage a Large Language Model (LLM) to guide the SS module in adaptively incorporating trend information to improve generalization. While in the interpretation process, we update the SS module to identify trend information for prediction and leverage the LLM's domain knowledge to provide intuitive natural language explanations. We conduct comprehensive experiments from qualitative and quantitative perspectives to demonstrate the SS module's superiority and plug-in benefits. Our code is included in the supplementary materials for examination and will be published upon acceptance.

Keywords Series saliency · Multivariate time series forecasting · Interpretable deep learning · Large language model

1 Introduction

Multivariate time series analysis has attracted widespread attention in various domains, such as air quality assessment (Zeger et al., 2000) and financial market price prediction (LeBaron et al., 1999). These applications often require predicting future values, making multivariate time series forecasting a promising research direction. For example, even predicting a

Editor: Georgiana Ifrim.

Extended author information available on the last page of the article

Published online: 07 October 2026

second or minute ahead can substantially contribute to generating returns in modeling stock market price (Sezer et al., 2020). Typically, multivariate time series data can be decomposed into three components: trend, seasonality, and residual components (Hamilton, 1994), with the trend components reflect the overall upward or downward patterns. However, existing methods (Jin et al., 2023; Tsay, 2013) often fail to effectively capture key trend information inherent in time series data, thereby limiting their ability to capture implicit correlations between feature and time dimensions in complex real-world scenarios.

The existing methods can be broadly categorized into statistical learning methods and deep learning methods. Traditional statistical methods (e.g., ARIMA (Box & Jenkins, 1976; Nelson, 1998)) have been used in forecasting tasks for decades. These methods often rely on linearity or stationary assumptions, making them easy-to-interpret but limited in predictive performance (Box & Jenkins, 1976; Harvey, 1990). Recently, deep learning methods, such as long-short term memory (LSTM) (Hochreiter & Schmidhuber, 1997), nonlinear autoregressive neural networks (NARNN) (Hassan et al., 2021), gated recurrent units (GRU) (Chung et al., 2014), Transformer (Vaswani et al., 2017) and TimeMixer (Wang et al., 2024), have outperformed traditional statistical learning methods by directly modeling nonlinear relationships in multivariate time series forecasting. Although these deep models have proven effective in improving accuracy, they often face challenges in interpretability (Castelvecchi, 2016), which hinders their applications in some high-stakes scenarios such as healthcare. In this regard, many interpretation methods have been developed in the fields of computer vision (Guo et al., 2022) and natural language processing (Galassi et al., 2020), yet limited in time series data due to disruptions of inherent trend patterns (e.g., sawtooth or spike anomalous patterns) (Ismail et al., 2020). Moreover, most deep learning methods tend to overfit local fluctuations in high-stakes scenarios while ignoring trend changes. Consequently, developing accurate and interpretable methods for multivariate time series forecasting is unexplored research topics, where key trend information poses new challenges.

To address the aforementioned challenges, we introduce the Series Saliency (SS) module, a versatile plug-in approach designed to capture essential trend information for better generalization and interpretability. Specifically, we transform multivariate time series data into a set of $window \times feature$ series images by coherently considering the time and feature dimensions. We propose the SS module to combine series images and the corresponding trend information using a learnable mask and theoretically demonstrate that it can effectively explore previously uncharted input spaces while preserving trend information. During training, we employ a Large Language Model (LLM)'s (Liu et al., 2024) domain knowledge to guide the SS module in adaptively incorporating trend information. While in the interpretation process, we update the SS module to identify trend information for prediction and leverage the LLM's domain knowledge to provide more intuitive insights. We further design a theoretically guaranteed regularization term to avoid spike-like anomalous patterns. These efforts enable the SS module to improve both the accuracy and the interpretability of deep models.

To empirically verify the effectiveness of the proposed SS module, we conduct extensive experiments on various multivariate time series forecasting tasks with different horizons, including air quality assessment and stock price prediction. The results show that the SS module significantly improves the performance of various deep models in a plug-in manner. Meanwhile, we provide qualitative and quantitative interpretability results to demonstrate that the SS module outperforms various competitive baselines and assists domain experts

in analyzing behavioral patterns. Further, we conduct detailed ablation studies to verify the effectiveness of each component within the SS module. The paper is a substantial extension of the preliminary conference paper (Pan & Hu, 2021), and a summary of changes can be found in Appendix A.

In summary, our key contributions are as follows.

- We propose the Series Saliency (SS) module, and theoretically verify that it can effectively captures key trend information while preserving the trend information.
- We leverage the Large Language Model (LLM)'s domain knowledge to guide the SS module to seamlessly incorporate trend information in the training process and update the SS module to identify key trend patterns while generating intuitive natural language explanations in the interpretation process.
- We conduct comprehensive experiments from qualitative and quantitative perspectives to demonstrate the SS module's superiority to improve generalization and interpretability in a plug-in manner.

The paper comprises the following sections: Sect. 2 reviews the related work. Section 3 presents the training and interpretation methods based on our proposed SS module. Section 4 presents experimental results that verify the SS module's effectiveness. Section 5 summarizes our key findings and discusses future research directions.

2 Related Work

In this section, we review previous work related to the proposed Series Saliency (SS) module. Section 2.1 introduces various methods for multivariate time series forecasting, while Sect. 2.2 introduces interpretability methods for multivariate time series data.

2.1 Multivariate Time Series Forecasting

Existing methods for multivariate time series forecasting can be broadly categorized into traditional statistical methods and deep learning based methods. Traditional statistical methods (e.g., Auto-Regressive Integrated Moving Average (ARIMA) (Box & Jenkins, 1976; Nelson, 1998)) are relatively easy-to-interpret and perform well on small-scale datasets. However, these methods typically rely on assumptions of linearity or stationarity, limiting their performance in non-linear data distributions of real-world scenarios (LeCun et al., 2015). For time series classification tasks, traditional statistical methods still achieve competitive performance on UCR/UEA datasets (Middlehurst et al., 2024). For univariate forecasting tasks, such as the M4 competition (Makridakis et al., 2020), traditional statistical models also perform well. For more challenging multivariate time series dataset, which includes dynamic relationships within multiple variables changing over time, it is relatively unexplored to combine classical statistical models with deep learning models for better generalization.

In recent years, deep learning based methods have shown strong performance in multivariate time series forecasting, including convolutional neural network (CNN) (Lea et al., 2016), long-short term memory (LSTM) (Hochreiter & Schmidhuber, 1997), gated recur-

rent units (GRU) (Chung et al., 2014), Transformer (Vaswani et al., 2017) and TimeMixer (Wang et al., 2024). Lai et al. (20218) introduces LSTNet, which combines CNN and RNN to capture temporal dependencies. Zhou et al. (2021) proposes Informer the Transformer-based architecture by leveraging the self-attention mechanism to effectively capture long-term patterns. Oreshkin et al. (2019) proposes N-BEATS based on Transformer architecture decomposes time series into trend and seasonal components to improve performance. TimeMixer (Wang et al., 2024), inspired by MLPMixer (Tolstikhin et al., 2021), integrates correlations across time and feature dimensions for predictions. Also, the long-running Makridakis (M) competitions (M1–M5) have become foundational empirical benchmarks for time series forecasting. The M1 (Makridakis et al., 1982) and M3 (Makridakis & Hibon, 2000) includes dozens of methods on thousands of series, and show that classical statistical approaches still serve as strong baselines. M4 (Makridakis et al., 2020) scales up to 100,000 (mostly univariate) series that span multiple frequencies, strengthening the value of combinations and hybrid methods. The most recent M5 competitions move to hierarchical retail sales and evaluate both point estimation (Makridakis et al., 2022a) and uncertainty quantification (Makridakis et al., 2022b). Although most deep models tend to overfit short-term fluctuations and often fail to effectively incorporate trend information, thereby limiting their performance in more challenging real-world scenarios. Although deep models have achieved promising results in multivariate time series forecasting, further exploration is needed to effectively leverage long-term trend information in time series data to improve accuracy.

2.2 Interpretation Methods in Multivariate Time Series Analysis

Interpretability has become another increasingly important topic in multivariate time series forecasting. Traditional statistical methods (e.g., Auto-Regressive Integrated Moving Average (ARIMA) (Box & Jenkins, 1976; Nelson, 1998)) provide insights by analyzing regression coefficients, residual distributions or covariance structures. While deep models outperform traditional statistical methods in predictive accuracy, the difficulty of interpreting their predictions limits their applications in high-stakes scenarios, such as financial market price forecasting. Some attempts have been made to apply general post-hoc interpretability methods to multivariate time series forecasting for analyzing behavioral patterns. Gradient-based saliency methods (Baehrens et al., 2010) compute the local gradient of the model output with respect to each input feature, so features with large gradient values are identified as key patterns. Propagation-based approaches like DeepLIFT (Shrikumar & Greenside, 2017) compare the activation of each neuron to a reference baseline and backpropagates the activation differences to assign contribution scores to input features. LIME (Ribeiro et al., 2016) involves slightly perturbing the target deep model and fitting a local linear model to approximate its behavior. Occlusion (Zeiler, 2014) systematically removes parts of the input and measures the change in the model score, revealing the regions or variables most important for prediction. Also, a similar feature ablation approach (Suresh et al., 2017) is used based on LSTM models for clinical data, removing individual input features to analyze deep model's behaviors. Additionally, Wang et al. (2018) decomposes time series data into various frequencies and analyzes the feature spectrum for prediction. However, these methods predominantly focus on local features at specific time steps, neglecting key trend information that spans multiple time steps. Another model-agnostic method is Permutation

(Molnar, 2020), which measures the increase in prediction error when features are randomly shuffled. Recently, CoMTE (Ates et al., 2021) generates counterfactual explanations by finding minimal changes via flipping to capture key features or shapelets for predictions.

Another avenue involves leveraging the attention mechanism from natural language processing (Galassi et al., 2020) and computer vision (Guo et al., 2022) to interpret time series data by assigning relative importance weights to different time steps. The attention values typically reflect the significance of different steps, and cannot explicitly capture the distribution of importance at a given time step (Chaudhari et al., 2021). Furthermore, Ismail et al. (2020) proposes a two-stage method that separates the time and feature dimensions for interpretation. Zhang et al. (2023) proposes ShapeTime based on shapley values, transforming learnable latent space into the frequency domain to provide stable temporal explanations. WinIT(Leung et al., 2023) removes a specific feature within a certain window and compare the change in the prediction to determine the contribution of feature to the prediction. However, these methods may produce a suboptimal solution as they ignore the implicit relationships between time and feature dimensions like trend information. Therefore, further exploration is needed to leverage trend information to improve the interpretability of deep models.

3 Our Method

In this section, we introduce the Series Saliency (SS) module, including a detailed implementation of the training and interpretation processes based on the SS module. Section 3.1 presents the setup and notations of multivariate time series forecasting. Section 3.2 provides the detailed implementations of the SS module. Section 3.3 presents the training process based on the Large Language Model (LLM)-guided SS module. Section 3.4 presents the interpretation process based on the SS module.

3.1 Problem Settings

A multivariate time series refers to a series of observed data at different time steps. Let S denotes the observed time series data as

$$S = [s_1, s_2, \cdots, s_t, \cdots] \tag{1}$$

where $s_t \in \mathbb{R}^D$ is the feature vector at time step t. Our goal is to predict the future value $s_{t+\tau}$ after a given horizon τ. The horizon is chosen according to the specific forecasting task. In addition to improving performance, we are also interested in identifying the feature importance score for predictions. An accurate and interpretable deep model can provide decision-makers with more valuable insights into the implicit correlations within features. However, developing accurate and interpretable deep learning methods is still challenging. Although many deep architectures (Che et al., 2018; Wan et al., 2019; Vaswani et al., 2017) often tend to overfit local fluctuations while ignoring key trend information. Thus, they may fail to identify critical features for predictions. So, it is crucial to dynamically incorporate trend information into both the training and interpretation processes for multivariate time series forecasting.

3.2 Series Saliency

To address the challenges, we propose the Series Saliency (SS) module, and theoretically demonstrate that it can perceive key trend information while exploring the previously uncharted sample space to improve accuracy and interpretability in training and interpretation processes.

3.2.1 Multivariate Time Series Forecasting

To coherently consider both the time/feature dimensions and drawing inspiration from the saliency maps (Dabkowski & Gal, 2017), we sequentially slide the window to collect changes from previous time steps (e.g., trend changes or seasonal effects), converting the multivariate time series data into a set of series images $X \in \mathbb{R}^{T\times D}$ to align with saliency-style operations as

$$X = \begin{bmatrix} x_{1,1} & x_{1,2} & \cdots & x_{1,D} \\ x_{2,1} & x_{2,2} & \cdots & x_{2,D} \\ \vdots & \vdots & & \vdots \\ x_{T,1} & x_{T,2} & \cdots & x_{T,D} \end{bmatrix}, \tag{2}$$

where D is the feature dimension at each time step, and T is the window size determined by the periodic properties of datasets (e.g., $T = 12$ hours for Air Quality dataset). The primary motivation is to highlight the $T \times D$ 2D matrix structure, which facilitates the definition of Series Saliency (SS) module. The representation intuitively aligns with the input format of Computer Vision models (Selvaraju et al., 2017). Thus, the implicit dynamic correlations between variables can be preserved in the format, due to each series image can be further decomposed into trend, seasonality, and residual components (Hamilton, 1994) And, we apply Gaussian smoothing (Wen et al., 2020) to each feature dimension, extracting trend information.

To each series image X_i corresponds to a target vector $y_i \in \mathbb{R}^D$ given horizon τ. Our goal is to learn a mapping function $f_\theta(X) : \mathbb{R}^{T\times D} \to \mathbb{R}^D$ parameterized by θ from the dataset $\mathcal{D} = \{(X_i, y_i)\}$ for multivariate time series forecasting.

In time series data, each sample can be decomposed into trend, seasonal, and residual components (Hamilton, 1994). Compared to the original X, the trend information $\hat{X} \in \mathbb{R}^{T\times D}$ is smoother. To effectively extract the trend information (Wen et al., 2020), we apply a Gaussian smoothing operation $\mathcal{G}_{\sigma^2}(\cdot)$ to each feature dimension, extracting trend $\hat{x}_{t,i}$ as

$$\hat{x}_{t,i} = \mathcal{G}_{\sigma^2}(x_{t,i}) = \sum_{k=-K}^{K} x_{t+k,i}\, \mathcal{G}_{\sigma^2}(k) = \sum_{k=-K}^{K} x_{t+k,i} \frac{1}{\sqrt{2\pi\sigma^2}} \exp\{-\frac{k^2}{2\sigma^2}\} \tag{3}$$

where σ^2 is the variance of isotropic noise, and K is the support region of the Gaussian blur function $\mathcal{G}_{\sigma^2}$. Although smoothing operation $\mathcal{G}_{\sigma^2}(\cdot)$ effectively removes high-frequency noise to extract key trend information $\hat{X}$, relying solely on a single scalar for its linear integration λ of this information (i.e., $(1-\lambda)X + \lambda\hat{X}$) without considering the varying significance of each time step, may introduce abnormal patterns. These patterns may mislead deep models into overfitting and ignoring overall trend information. Therefore, it is crucial

to adaptively incorporate trend information in different time steps (e.g., using a Mixup-like strategy (Zhang et al., 2017a)) to mitigate the risk of incorporating anomalies.

To address the aforementioned challenges, we replace the single scalar λ with a learnable mask $M \in [0,1]^{T\times D}$ to adaptively combine the original X with its corresponding trend information $\hat{X} \in \mathbb{R}^{T\times D}$. Formally, the SS module is defined as

$$\tilde{X} = M \bigodot \hat{X} + (E - M) \bigodot X \tag{4}$$

where E is a matrix of ones with the same shape as X, and $\odot$ denotes the element-wise operation. When $m_{t,i}$ is close to 1, the i-th feature at time step t aligns with the trend information. When $m_{t,i}$ is close to 0, it aligns with the original series image X. As shown in Fig. 1 b and Theorem 1, the SS module can incorporate trend information while flexibly exploring previously uncharted sample space. Also, Theorem 1 formalizes that the SS module can perform element-wise interpolation $\tilde{X}$ between the original X and trend information $\hat{X}$, so the SS module constructs a feasible convex set to enable safe interpolation, which possesses significant potential to integrate trend information for better predictive performance. Furthermore, Theorem 1 also demonstrates the "non-expansiveness": for any p-norm, we have $\|\tilde{X} - X\|_p \leq \|\hat{X} - X\|_p$ to enhance training stability when utilizing augmented samples based on the SS module.

However, a simplistic integration of trend information may unintentionally weaken crucial trend signals, leading deep models to overfit local fluctuations. As suggested by (Gamboa, 2017), statistical properties (e.g. mean, change points, and volatility) can serve as guiding indicators to help deep models differentiate between "stationary regions" and "volatile regions." Building on this insight, we propose to utilize these statistical properties as external supervisory signals for the SS module, enabling it to effectively incorporate trend information while capturing both local fluctuations and the broader trend dynamics.

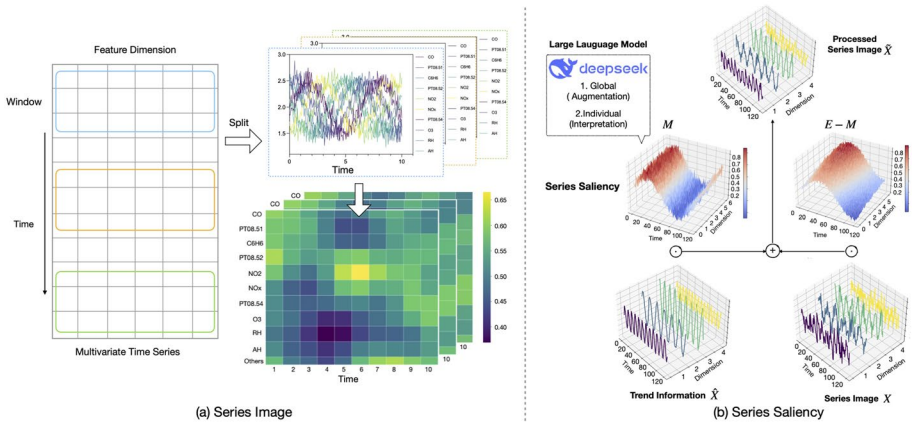


Fig. 1 (Left): Multivariate time series and the corresponding feature importance map, where we sequentially slide the window for collecting changes from previous time steps to extract series images from multivariate time series, and each series image is associated with a feature importance map to identify informative features for prediction; (Right): Series Saliency (SS) module incorporates trend information within time series data and adaptively enhances key trend patterns with Large Language Model (LLM). Best viewed in color

Theorem 1 *Consider the series image $\tilde{X}$ formed by the original sample X and the trend information $\hat{X}$. The processed series image $\tilde{X}$ constitutes a convex set, and each feature $\tilde{x}_{t,i}$ satisfies*

$$\min(x_{t,i}, \hat{x}_{t,i}) \leq \tilde{x}_{t,i} \leq \max(x_{t,i}, \hat{x}_{t,i}) \tag{5}$$

In other words, $\tilde{x}_{t,i}$ remains within the range of $\hat{x}_{t,i}$ and $x_{t,i}$, while exploring the sample space. Please refer to the proof details in Appendix B.1.

3.3 Training Process

To more effectively capture key trend patterns, we combine LLM's domain knowledge with statistical properties to guide the SS module and add an additional Auto-Regressive (AR) component into the Deep Learning (DL) component to improve generalization.

3.3.1 LLM-Guided Series Saliency Module

Although the SS module has great potential in incorporating trend information, the implicit correlations between feature and time dimensions may lead to blind combinations by the train-from-scratch strategy (Dabouei et al., 2020; Pan & Hu, 2021), which causes deep models to overfit local fluctuations. In this regard, we leverage LLM's domain knowledge to guide the SS module. Recent studies have shown that LLM achieves remarkable performance in various time series analysis tasks, like forecasting (Gruver et al., 2024; Jin et al., 2023). Based on this observation, we use LLM's domain knowledge together with statistical properties of time series data to identify "stationary regions" and "volatile regions", which guides the SS module in generating $M^{(\mathrm{LLM})} \in \mathbb{R}^{T \times D}$. The $M^{(\mathrm{LLM})}$ adjusts the contribution of the trend information $\hat{X}$ and the original series image X at each feature and time step to generate $\tilde{X}^{(\mathrm{LLM})}$ as

$$\tilde{X}^{(\mathrm{LLM})} = M^{(\mathrm{LLM})} \odot \hat{X} + \left(E - M^{(\mathrm{LLM})}\right) \odot X, \tag{6}$$

where $\odot$ denotes element-wise operation. We construct the prompt P in Fig. 3 as input for the LLM to generate a more effective $M^{(\mathrm{LLM})}$. Based on the feedback from the LLM, we can obtain $M^{(\mathrm{LLM})}$ and its corresponding interpretations for further analysis.[1] To ensure robust generalization across datasets without the need for domain-specific re-engineering, we adopt a strategy that combines a "unified prompt template" with "dynamic statistical information", as shown in Fig. 3. Specifically, while the structure of the prompt template remains consistent across all experiments, we dynamically inject data-specific statistical properties, including the mean, standard deviation, and volatility of both the original time series data and trend components, into the prompt. These statistical features provide a critical domain-specific context (e.g., capture the seasonality in electricity), allowing the LLM to adaptively guide the SS module.

[1] In high-noise environments (e.g., financial market forecasting), selectively amplifying critical trend information becomes imperative to prevent models from overfitting to short-term noises.

3.3.2 Auto-regressive Component

To mitigate deep learning models' sensitivity to short-term fluctuations, we introduce an additional Auto-Regressive (AR) component to explicitly enhance overall trend information. As shown in Fig. 2, the original series image X is converted to $\tilde{X}^{(\mathrm{LLM})}$ based on the LLM-guided SS module, and then $\tilde{X}^{(\mathrm{LLM})}$ is fed into both the DL and AR components. Then we obtain the prediction $\hat{y}^{(\mathrm{DL})}$ by feeding $\tilde{X}^{(\mathrm{LLM})}$ into the DL component, which can be replaced by other powerful deep models, such as RNN (Che et al., 2018; Hochreiter & Schmidhuber, 1997), TCN (Wan et al., 2019), and Transformer (Vaswani et al., 2017), LSTNet (Lai et al., 20218), TimeMixer (Wang et al., 2024)), etc. For the AR component, the prediction $\hat{y}^{(\mathrm{AR})}$ is defined as

$$\hat{y}^{(\mathrm{AR})} = \sum_{j=1}^{p} W_j^{(\mathrm{AR})\top} y_{t-k} + b^{(\mathrm{AR})}, \tag{7}$$

where p is the order of the AR model. The initial p can be determined by datasets with periodic patterns (e.g., $p = 12$ for the hourly Air Quality (Kolehmainen et al., 2001)) or by cross-validation on the validation sets. Then, the final prediction is obtained by combining both $\hat{y}^{(\mathrm{AR})}$ and $\hat{y}^{(\mathrm{DL})}$ as

$$\hat{y} = \hat{y}^{(\mathrm{AR})} + \hat{y}^{(\mathrm{DL})}. \tag{8}$$

Thus, the final prediction $\hat{y}$ produced by integrating the DL and AR components enables deep model to effectively capture short-term fluctuations in multivariate time series data. Ablation studies show that both the DL and AR components are essential to improve overall performance. The dual-path architecture enables the DL component to capture both local fluctuations and trend information.

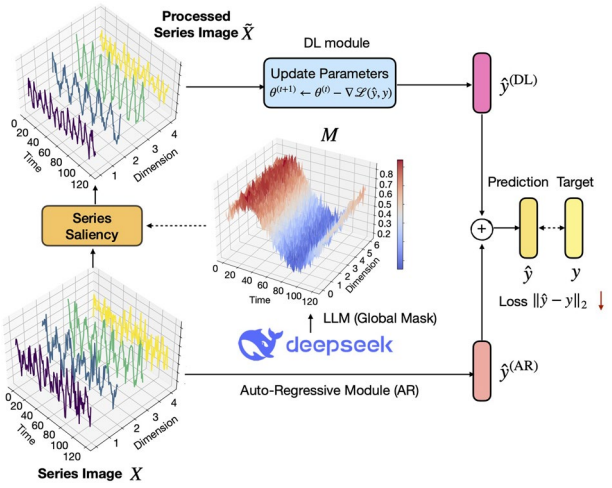


Fig. 2 The training process based on the large language model guided-series saliency module, whose augmented samples are fed into the deep learning (DL) component and auto-regressive (AR) component, respectively. Best viewed in color

3.3.3 Optimization Process Based on SS Module

In this section, we present the general training process based on the LLM-guided SS module. Formally, we train a deep model parameterized by the learnable $\phi = \{W^{(\mathrm{AR})}, b^{(\mathrm{AR})}, \theta\}$ in the training set $\mathcal{D} = \{(X_i, y_i)\}$, where y_i is the true target vector given a fixed horizon τ , $W^{(\mathrm{AR})}$ and $b^{(\mathrm{AR})}$ are the weight and bias of the AR component in Eq. (7), θ denotes the parameter of the DL component and M is the parameter of the SS module. Before training, we collect statistics of samples (e.g., mean, volatility, and change points) and construct prompts P as input for the LLM to produce $M^{(\mathrm{LLM})}$, which guides the SS module to use trend information. At each epoch, we sequentially collect a mini-batch $\{X_i, y_i\}_{i=1}^{B}$ from the training set. For each sample (X_i, y_i) in the mini-batch, we use Eq. (6) to obtain $\tilde{X}_i^{(\mathrm{LLM})}$ and $M^{(\mathrm{LLM})}$ by incorporating LLM's domain knowledge. Then, we feed the original sample into the AR component for $\hat{y}_i^{(\mathrm{AR})}$. These two components are combined via Eq. (8) to obtain the final prediction $\hat{y}_i$. We accumulate the mean squared error loss $\|\hat{y}_i - y_i\|_2$ into $\mathcal{L}_1(\phi, \mathcal{D})$. Additionally, we feed the original series image X directly into the DL component (surpass the SS module), and combine it with the AR component for another prediction $\hat{y}_i'$. The term $\|\hat{y}_i' - y_i\|_2$ is accumulated into the loss function $\mathcal{L}_1(\phi; \mathcal{D})$. Therefore, the overall objective function is defined as

$$\mathcal{L}_1(\phi; \mathcal{D}) = \sum_{(x_i, y_i) \in \mathcal{D}} \|\hat{y}_i - y_i\|_2 + \sum_{(x_i, y_i) \in \mathcal{D}} \|\hat{y}_i' - y_i\|_2. \tag{9}$$

Then we use stochastic gradient descent to update the learnable parameter set ϕ. The LLM-guided SS module helps the deep model perceive trend information, while the AR component helps the deep model capture short-term fluctuations to significantly improve generalization on real-world data.

3.4 Interpretation Process

Interpretability is another crucial aspect of multivariate time series forecasting, particularly in high-stakes scenarios where domain experts are involved (Chakraborty et al., 2017). Recent work indicates that deep models achieves competitive performance in various tasks (Wen et al., 2020). However, these methods are difficult to interpret, which may hinder their applications in some real-world scenarios like healthcare and financial decision-making. Zhang et al. (2021) indicates that deep models with better performance tend to effectively capture essential features. Hence, we design our interpretation method based on the SS module, which can highlight meaningful feature representations closely aligned with human perception and experience. Building on this foundation, we raise some key questions to further explore the role of trend information in feature extraction and model interpretation. Specifically, we raise some questions such as "Can using trend information more effectively extract key features?" and "Does trend information reveal learning patterns in deep models for multivariate time series forecasting?". Our empirical results in Sect. 4.3 show that the interpretation method based on the SS module effectively extracts key trend information, while leveraging the LLM's domain knowledge to provide natural language explanations.

3.4.1 Interpretation Process Based on SS Module

In the interpretation process, our goal shifts from "improving accuracy" to "identifying the most critical trend patterns", which involves conducting saliency analysis to evaluate the contributions of each time step and feature to the final prediction. Specifically, we fix all trained parameters (including DL and AR components) and only update the parameter M in the SS module to identify the key trend patterns. Meanwhile, we introduce a second-order difference regularization term $\ell_m(\cdot)$ into the objective function to avoid introducing anomalous patterns.

As shown in Fig. 4, we interpret a series image pair (X^*, y^*) from the test set given a horizon τ. With the optimal model parameters $\phi^* = \{W^{(\mathrm{AR})*}, b^{(\mathrm{AR})*}, \theta^*\}$ including both DL and AR components, we feed $\tilde{X}^*$ into both the DL and AR components and compute the squared error $\|\hat{y}^* - y^*\|_2$. The overall regularized objective function for the sample pair (X^*, y^*) is

$$\mathcal{L}(M; (X^*, y^*)) = \|\hat{y}^* - y^*\|_2 + \ell_m(M), \tag{10}$$

where the second-order difference regularization term $\ell_m(M)$ is used to distinguish between "series image" and "natural image". Note that there is a key difference between our "series images" and "natural images" in computer vision—row permutations (i.e., features) of series images do not affect the content, while switching rows of a nature image would destroy its semantic meaning. The mask M for the series image should preserve trend information inherent in time series data, whereas the natural image is sensitive to temporal order, which can destroy its semantic meaning (Szeliski, 2022). Thus, $\ell(M)$ should guide M to avoid anomalous patterns in $\tilde{X}$ as

$$\ell_m(M) = \sum_{t=1}^{T} \sum_{i=1}^{D} \|m_{t+1,i} - m_{t,i}\|_2^2, \tag{11}$$

where $m_{t,i}$ is the feature importance weight for the i-th feature at time step t. Then we use backpropagation to compute gradients and apply stochastic gradient descent to **only** update M. After convergence, the optimal M^* for the pair (X^*, y^*) identifies the key trend patterns for prediction. Theorem 2 theoretically demonstrates that the regularization term $\ell_m(M)$ guides the SS module in avoiding anomalous patterns to destroy trend information while satisfying the theoretical upper bound.

Although the optimal M^* offers valuable insights, it can still be difficult to interpret the model's logic directly, especially for non-expert users of deep learning. Therefore, we leverage LLM's domain knowledge to provide natural language explanations for deeper insights. Specifically, we combine the feature importance map with prediction as contextual information, and leverage LLM's domain knowledge to analyze key trend information. We construct the prompt P in Fig. 5 as input for LLM. Based on the LLM's feedback and the SS module, we can obtain the natural language explanation that helps non-experts make preliminary assessments, while the experts can also obtain a clearer view of the model's logic. Our interpretation methods based on the SS module increases confidence in the applications of artificial intelligence for multivariate time series forecasting, and facilitate further exploration of developing accurate and interpretable methods in some high-stakes scenarios.

3.4.2 Theoretical Analysis in Interpretation Process

In this section, we conduct an in-depth theoretical analysis to demonstrate that the regularization term $\ell_m(M)$ guides $\tilde{X}$ to avoid spike-like anomalous patterns in Theorem 2.

Theorem 2 *Suppose*

$$\sum_{t=1}^{T}\sum_{i=1}^{D}\left(m_{t+1,i}-m_{t,i}\right)^2 \leq \varepsilon, \tag{12}$$

where error $\epsilon > 0$ *and assume that the original sample X is relatively smooth, that each feature satisfies*

$$\left|x_{t+1,i}-x_{t,i}\right| \leq C_x, \quad C_x > 0. \tag{13}$$

Then for the corresponding series image $\tilde{X}_{t,i}$*, each element satisfies the theoretical bound*

$$|\tilde{x}_{t+1,i}-\tilde{x}_{t,i}| \leq \left[\sqrt{\epsilon}\cdot(K(K-1)+\alpha)+\alpha+1\right]C_X, \tag{14}$$

where $0 \leq \alpha \leq 1$*, and K represents the receptive field of* $\mathcal{G}_{\sigma^2}(\cdot)$ *and* $0 \leq \alpha \leq 1$*. Please refer to Appendix B.2 for details of the proof.*

4 Experiments

In this section, we describe the experimental setup and then analyze the experimental results. Section 4.1 presents the experimental setup, including datasets, evaluation metrics, deep models, and hyperparameter selection. Section 4.2 evaluates performance by incorporating the SS module in several real-world datasets. Section 4.3 presents both qualitative and quantitative results from various interpretation methods.

4.1 Experimental Setup

4.1.1 Datasets

We use four widely used regularly-sampled multivariate time series datasets, including *Air Quality*, *Industry*, *Exchange Rate*. The statistical properties of datasets are shown in Appendix C.1. All of these datasets are widely used in literature (Lai et al., 20218; Cui et al., 2020; Shih et al., 2019), which are split into 60% for training, 20% for validation, and 20% for testing. The complexity of these datasets ranges from easy to difficult. The evaluation datasets provide a standardized means of assessing the performance of different deep models used in Fig. 3 in multivariate time series forecasting. Additionally, Table 1 summarizes the key statistical properties used in Fig. 3.

- *Air quality* (Cui et al., 2020) contains 9358 instances of hourly averaged responses from five metal chemical sensors. The researchers collect data from March 2004 to February 2005 (approximately 1 year) in heavily polluted regions such as Beijing and London.

Please use these features to generate a dynamic weight distribution M that adaptively incorporates trend information for multivariate time series forecasting. Ensure you meet the following requirements:

1. If $m_{t,i} \in M$ is close to 1, then the i-th feature at time step t aligns more closely with the trend information.
2. If $m_{t,i} \in M$ is close to 0, it aligns more closely with the original series X.
3. Nonlinear functions (e.g., `sigmoid` or `tanh`) may be used to prevent abrupt weight changes.
4. Incorporate local statistical features (e.g., (`mean_X`, `std_X`, `mean_T`, `std_T`, `volatility_X`, `volatility_T`)) to distinguish between stable and volatile regions.
5. Return the results in `JSON` format, including:
   (a) `"M"`: a 2D array of shape (T, D), where T is the time dimension and D is the feature dimension.
   (b) `"explanation"`: interpret how M is generated (e.g., which features are utilized, and how change points are used to determine the contribution).

Return LLM-guided Mask ({`M`})

Fig. 3 The prompt P provided to the LLM during training

- *Industry* (Wei et al., 2020) consists of 3205 daily records, including Hang Seng Composite Index and 11 industry stock indices from the Wind platform. These indices cover various sectors such as consumer manufacturing, consumer services, energy, finance, industry, IT, conglomerates, commodities, real estate, and utilities. These records are collected from September 2006 to September 2019, covering 13 years.
- *Exchange rate* link (Shih et al., 2019; Lai et al., 20218) contains daily Exchange Rates, including Australia, the United Kingdom, Canada, Switzerland, China, Japan, New Zealand, and other countries. These records range from January 1990 to September 2016. It offers valuable insights into trends, volatility, and correlations among different currencies during this period, making it an essential resource for economic analysis, financial modeling, and time series forecasting.
- *Weather* link is (Oikolab, 2025) from the Monash Time Series Forecasting repository (Godahewa et al., 2021), which is a meteorological subset collected from the Oikolab weather data platform. The dataset provides hourly observations collected near Monash University, Clayton (Victoria, Australia) from January 2010 to May 2021, and comprises eight meteorological variables (e.g., air temperature, dew point, wind speed, sea-level pressure, relative humidity, cloud cover) organized as a multivariate time series for training and evaluating weather forecasting models.

To further evaluate the performance of SS module on irregularly sampled time series data, we conduct experiments on the Beijing Multi-Site Air Quality dataset (Zhang et al., 2017b), which contains hourly metrics from 2013 to 2017 at various monitoring stations. We select two representative stations: Aotizhongxin (urban center) and Changping (suburban area). Following settings in (Cao et al., 2018; Chen et al., 2022; Du et al., 2023), we simulate irregular sampling scenarios by randomly masking 10% and 30% of the time steps from the original dataset, representing various irregularly sampled dataset.

Table 1 The data-specific statistical properties of both the original time series data and trend components

Metric	Air_quality	Exchange rate	Weather	Multiple_industry
mean_X	[2.09, 1097.40, 10.00, 938.00, 112.40, 832.30, 1455.90, 1020.70, 49.20, 1.03, 235.60]	[0.78, 1.63, 0.82, 0.85, 0.14, 0.01, 0.65, 0.67]	[14.60, 9.49, 3.87, 101615.40, 0.74, 179.10, 323.80, 0.58]	[3072.10, 4484.40, 3727.00, 10678.60, 3347.00, 1331.70, 4599.90, 2410.50, 7913.80, 3016.70, 1748.90, 6008.00]
std_X	[1.32, 212.40, 7.31, 261.50, 44.00, 249.70, 339.40, 390.80, 17.00, 0.40, 195.40]	[0.14, 0.16, 0.12, 0.17, 0.02, 0.00, 0.12, 0.08]	[5.52, 3.59, 2.05, 969.30, 0.15, 261.70, 32.70, 0.36]	[540.20, 874.70, 803.60, 2745.80, 513.20, 280.00, 3611.50, 424.80, 3491.20, 617.40, 336.20, 1216.30]
mean_T	[2.09, 1097.20, 10.00, 938.10, 112.40, 832.20, 1456.10, 1020.50, 49.20, 1.03, 235.70]	[0.78, 1.63, 0.82, 0.85, 0.14, 0.01, 0.65, 0.67]	[14.60, 9.49, 3.87, 101615.50, 0.74, 179.10, 323.80, 0.58]	[3074.40, 4490.80, 3732.50, 10701.30, 3349.00, 1330.80, 4591.50, 2412.40, 7930.80, 3016.90, 1752.30, 6011.10]
std_T	[0.72, 136.70, 3.99, 145.10, 28.80, 167.10, 268.50, 266.00, 12.70, 0.38, 139.40]	[0.14, 0.16, 0.12, 0.17, 0.02, 0.00, 0.12, 0.08]	[4.63, 3.37, 1.66, 846.40, 0.10, 95.00, 27.50, 0.26]	[530.90, 863.20, 793.40, 2702.30, 502.60, 275.10, 3599.00, 420.40, 3469.50, 607.10, 328.40, 1203.50]
vol_X	[0.79, 102.80, 4.18, 135.40, 23.90, 108.80, 143.50, 176.90, 4.63, 0.05, 91.90]	[0.01, 0.01, 0.00, 0.01, 0.00,6.50×10^{-5}, 0.00, 0.00]	[1.08, 0.49, 0.48, 277.90, 0.04, 85.80, 9.79, 0.14]	[43.50, 60.10, 59.20, 227.40, 50.80, 23.70, 101.90, 31.20, 193.50, 49.10, 31.50, 60.40]
vol_T	[0.05, 7.50, 0.27, 9.15, 1.46, 9.30, 9.88, 15.20, 0.54, 0.01, 6.75]	[0.00, 0.00, 0.00, 0.00, 0.00,1.30×10^{-5}, 0.00, 0.00]	[0.15, 0.11, 0.10, 43.80, 0.01, 4.53, 1.36, 0.02]	[8.98, 12.80, 12.70, 46.40, 10.30, 5.52, 20.40, 6.52, 46.10, 10.50, 6.03, 11.60]

4.1.2 Models

We select two classical statistical models and five widely used deep models that typically achieve strong performance in multivariate time series forecasting:

- *Vector auto-regression (VAR)* (Lütkepohl, 2013; Sims, 1980) is a classical statistical multivariate time series model that captures linear interdependencies among multiple variables via their own and cross-variable lagged values, and is commonly used for multivariate time series forecasting.
- *Vector auto-regressive moving average (VARMA)* (Düker et al., 2025; Tsay, 2013) is a classical extension of the ARMA model to multiple time series, combining Auto-Regressive (AR) and Moving Average (MA) components to capture both cross-series dependencies and short-term fluctuations. VARMA offers a more flexible approach to multivariate time series forecasting.
- *Convolutional neural network (CNN)* (Lea et al., 2016) is a 7-layer temporal convolutional network that extracts local patterns along the time dimension via convolutional operations.
- *Gated recurrent unit (GRU)* (Hochreiter & Schmidhuber, 1997) is widely used to capture temporal dependencies for prediction. We also use the attention mechanism to focus on the most relevant hidden states for prediction.
- *LSTNet* (Lai et al., 20218) combines CNN and RNN. CNN captures local fluctuations, while RNN with skip connections identifies periodic patterns along the time dimension.
- *Transformer* (Vaswani et al., 2017) stacks L layers of the multi-head self-attention mechanism and position-wise layers to capture long-term trend information.
- *TimeMixer* (Wang et al., 2024) inspired by MLPMixer (Tolstikhin et al., 2021), fuses features across both time and feature dimensions for predictions.

4.1.3 Hyperparameter Settings

We tune the hyperparameters in the validation set to achieve optimal performance. We adapt Adam optimizer (Kingma & Ba, 2014) with a learning rate of 10^{-4} and use a batch size of 64 or 128. For pre-processing, we scale the data in the [0, 1] range to avoid extreme values and ensure numerical stability. We employ the Deepseek-v3 Large Language Model (Liu et al., 2024) to guide the SS module in both the training and interpretation processes. We carefully review the public available materials for DeepSeek-V3. These sources describe a pre-training corpus of 14.8T tokens with a higher proportion of mathematical and programming samples and broader multilingual coverage, and they does not explicitly list specific time series datasets used in our paper, such as Air quality, Industry, Exchange rate and Weather. To further minimize any risk of data leakage, we only collects statistics of samples (e.g., mean, volatility, and change points) to construct a training prompt in Fig. 3 as input for LLM, without including any raw time series segments and specific data identifiers.

4.1.4 Evaluation Metrics

We use three representative metrics in (Lai et al., 20218; Cui et al., 2020) for multivariate time series forecasting, including *Relative Squared Error (RSE)*, *Relative Absolute Error*

(RAE), *Empirical Correlation Coefficient (CORR)* and their relationships to *RootMSE/MAE*. RSE and RAE normalize prediction errors via dividing them by the error of a naive mean predictor, enabling fair comparisons across datasets with different scales. On the other hand, RootMSE/MAE are scale-dependent metrics without normalization, and their values increase proportionally with dataset dimensionality, so they are inappropriate for cross-dataset fair comparisons.

- *Relative squared error (RSE)* measures the squared error of predictions relative to the standard deviation of the target values as

$$\mathrm{RSE} = \frac{\sqrt{\frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2}}{\mathrm{std}(Y)} = \frac{\mathrm{RMSE}}{\mathrm{std}(Y)}, \tag{15}$$

where $\hat{y}_i$ and y_i are the prediction and target value, and $\mathrm{std}(Y)$ is the standard deviation of the entire test set. A lower RSE value indicates better performance. RSE equals RootMSE/$\mathrm{std}(Y)$ and removes scale effects for fair comparison across different scales datasets.

- *Relative absolute error (RAE)* measures the absolute error of predictions relative to the average absolute deviation of the target values as

$$\mathrm{RAE} = \frac{\sqrt{\frac{1}{N}\sum_{i=1}^{N}|y_i - \hat{y}_i|}}{\frac{1}{N}\sum_{i=1}^{N}|y_i - \bar{y}|}, \tag{16}$$

where $\bar{y}$ is the mean of the target values. A lower RAE indicates that the prediction error is small compared to the variation in the dataset.

- *Empirical correlation coefficient (CORR)* quantifies the correlation between the target values and prediction as

$$\mathrm{CORR} = \frac{\sum_{i=1}^{N}(\hat{y}_i - \bar{y})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^{N}(\hat{y}_i - \bar{y})^2 \sum_{i=1}^{N}(y_i - \bar{y})^2}}, \tag{17}$$

where y and $\hat{y}$ are the target value and prediction. $\bar{y}$ is the mean of target values. A higher CORR value indicates a stronger correlation between the predictions and the target values.

4.1.5 Experimental Platform

We introduce the harware configuration including GPU, CPU, RAM, etc. Our experimental platform includes two NVIDIA A10 GPUs with 24 GB video memory, two Intel Xeon Gold 5416 S CPUs (16 cores/32 threads) and two Micron Technology MTC40F2046S1RC48BAX 128GB memory modules. The experimental platform enables efficient training, evaluation, and data processing for conducting all experiments reported in this work.

4.2 Experimental Results on Overall Performance

4.2.1 Overall Results

Regularly Sampled Dataset Table 2, Table 3 and Table 4 present the overall performance of various models on three datasets with different horizons. The observations show that the LLM-guided SS module with AR component significantly improves the performance of various models in a plug-in manner to improve generalization, particularly for larger τ . This is because the complexity increases with the horizon τ, the LLM's domain knowledge enables models to capture key trend information, while the AR component captures short-term fluctuations. These observations show that among all models in comparison, Transformer (Vaswani et al., 2017) achieves the best performance in most settings due to the powerful representation capability of the self-attention mechanism. Also, the overall performance of classical statistical models remains below Transformer. Therefore, we use the Transformer as the main architecture for further analysis.

4.2.2 Ablation Study

We conduct ablation studies to demonstrate the effectiveness of each component, including the Series Saliency (SS) module and the Auto-Regressive (AR) component.

- *w/o SS*: The model in Fig. 2 without SS module.
- *w/o LLM*: The model in Fig. 2 with the SS module, but without the guidance of LLM.
- *w/o AR*: The model in Fig. 2 with the LLM-guided SS module but without the AR component.
- *w/SS*: The model in Fig. 2 with the LLM-guided SS module and AR component.

Fig. 6 (a) shows the performance of Transformer on the Air Quality dataset ($\tau = 3$). Due to the space limit, additional ablation studies for the horizon $\tau = 6$ and $\tau = 12$ can be found in Appendix C.2. We do not use any perturbations in *w/o SS*. The results indicate that both the AR component and the guidance of LLM contribute significantly to performance improvements. Removing either component leads to degenerated performance.

We further evaluate the computational costs of different deep models using SS (i.e., *w/ SS*) or not (i.e., *w/o SS*). Figure 6 (b) shows that there is minimal difference in time between *w/SS* and *w/o SS*. The network latency associated with API calls is the dominant computational cost. We need to clarify that computational overhead reported in Fig. 6 (b) represents the total additional time required to generate augmented samples for the entire dataset, rather than a per instance cost or online inference latency. Specifically, the generation process, which leverages the LLM's domain knowledge, is conducted entirely offline as a preprocessing step prior to the training phase. Therefore, this additional time cost is incurred only once during data preparation. Consequently, the SS module significantly improves predictive performance without introducing any additional computational overhead during the inference stage, making it highly suitable for multivariate time series forecasting applications with limited computational resources.

Additionally, we conduct more experiments to compare the performance of variants including naive interpolations on the Exchange-rate dataset.

Table 2 The RSE values of various methods on air quality, industry, exchange rate and weather datasets when horizon $\tau \in \{3, 6, 12\}$

Methods	Air quality			Industry			Exchange_rate			Weather		
	τ=3	τ=6	τ=12	τ=3	τ=6	τ=12	τ=3	τ=6	τ=12	τ=3	τ=6	τ=12
VAR	0.356 ± 0.001	0.393 ± 0.001	0.373 ± 0.001	0.937 ± 0.001	0.943 ± 0.001	0.667 ± 0.001	0.440 ± 0.001	0.399 ± 0.001	0.445 ± 0.001	0.126 ± 0.001	0.132 ± 0.001	0.149 ± 0.001
+Ours	**0.353** ± 0.001	**0.385** ± 0.001	**0.359** ± 0.001	**0.539** ± 0.001	**0.550** ± 0.001	**0.602** ± 0.001	**0.397** ± 0.001	**0.298** ± 0.001	**0.286** ± 0.001	**0.117** ± 0.001	**0.122** ± 0.001	**0.123** ± 0.001
Gains	0.003	0.008	0.014	0.398	0.393	0.065	0.043	0.101	0.159	0.009	0.010	0.026
VARMA	0.376 ± 0.001	0.411 ± 0.001	0.415 ± 0.001	0.828 ± 0.001	0.823 ± 0.001	0.804 ± 0.001	0.472 ± 0.001	0.467 ± 0.001	0.364 ± 0.001	0.131 ± 0.001	0.134 ± 0.001	0.136 ± 0.001
+Ours	**0.365** ± 0.001	**0.400** ± 0.001	**0.398** ± 0.001	**0.819** ± 0.001	**0.819** ± 0.001	**0.798** ± 0.001	**0.363** ± 0.001	**0.363** ± 0.001	**0.339** ± 0.001	**0**.096 ± 0.001	**0.105** ± 0.001	**0.120** ± 0.001
Gains	0.011	0.011	0.017	0.009	0.004	0.006	0.109	0.104	0.025	0.035	0.029	0.016
CNN	0.523 ± 0.001	0.557 ± 0.003	0.526 ± 0.002	0.585 ± 0.001	0.594 ± 0.001	0.588 ± 0.001	0.321 ± 0.001	0.326 ± 0.001	0.331 ± 0.001	0.274 ± 0.137	0.274 ± 0.137	0.266 ± 0.138
+Ours	**0.437** ± 0.023	**0.446** ± 0.003	**0.481** ± 0.003	**0.447** ± 0.014	**0.436** ± 0.005	**0.458** ± 0.021	**0.255** ± 0.001	**0.257** ± 0.001	**0.259** ± 0.001	**0.123** ± 0.052	**0.126** ± 0.048	**0.126** ± 0.046
Gains	0.086	0.111	0.045	0.138	0.158	0.130	0.066	0.069	0.072	0.151	0.148	0.140
RNN	0.343 ± 0.001	0.408 ± 0.000	0.384 ± 0.001	0.921 ± 0.001	0.934 ± 0.001	0.969 ± 0.001	0.282 ± 0.000	0.284 ± 0.000	0.283 ± 0.001	0.343 ± 0.254	0.345 ± 0.253	0.371 ± 0.258
+Ours	**0.304** ± 0.001	**0.391** ± 0.000	**0.367** ± 0.001	**0.652** ± 0.001	**0.606** ± 0.001	**0.558** ± 0.001	**0.125** ± 0.001	**0.127** ± 0.001	**0.132** ± 0.001	0.316 ± 0.159	0.311 ± 0.162	0.317 ± 0.162
Gains	0.039	0.017	0.017	0.269	0.328	0.411	0.157	0.157	0.151	0.027	0.034	0.054
LSTNet	0.357 ± 0.003	0.401 ± 0.002	0.400 ± 0.004	0.661 ± 0.001	0.676 ± 0.001	0.703 ± 0.001	0.318 ± 0.001	0.312 ± 0.001	0.319 ± 0.001	0.443 ± 0.065	0.441 ± 0.072	0.451 ± 0.070
+Ours	**0.348** ± 0.006	**0.377** ± 0.002	**0.370** ± 0.001	**0.504** ± 0.007	**0.501** ± 0.004	**0.518** ± 0.006	**0.126** ± 0.001	**0.156** ± 0.001	**0.186** ± 0.001	0.097 0.025	0.095 ± 0.023	0.096 ± 0.023
Gains	0.009	0.024	0.030	0.157	0.175	0.185	0.192	0.156	0.133	0.346	0.346	0.355
Timemixer	0.478 ± 0.005	0.539 ± 0.004	0.516 ± 0.002	0.619 ± 0.001	0.614 ± 0.001	0.598 ± 0.001	0.333 ± 0.001	0.334 ± 0.001	0.342 ± 0.001	0.153 ± 0.051	0.209± 0.054	0.173± 0.034
+Ours	**0.414** ± 0.006	**0.461** ± 0.011	**0.456**± 0.007	**0.412**± 0.002	**0.423**± 0.003	**0.437**± 0.009	**0.306** ± 0.001	**0.325**± 0.001	**0.336**± 0.001	0.140 ± 0.001	0.133 ± 0.001	0.096 ± 0.001

Table 2 (continued)

Methods	Air quality			Industry			Exchange_rate			Weather		
	τ=3	τ=6	τ=12	τ=3	τ=6	τ=12	τ=3	τ=6	τ=12	τ=3	τ=6	τ=12
Gains	0.064	0.078	0.060	0.207	0.191	0.161	0.027	0.009	0.006	0.013	0.076	0.077
Transformer	0.354 ± 0.039	0.367 ± 0.001	0.377 ± 0.001	0.451 ± 0.001	0.458 ± 0.001	0.457 ± 0.001	0.211 ± 0.001	0.213 ± 0.001	0.223 ± 0.010	0.026 ± 0.065	0.149 ± 0.061	0.023 ± 0.061
+Ours	**0.340** ± 0.001	**0.355** ± 0.001	**0.365** ± 0.001	**0.405** ± 0.001	**0.376** ± 0.001	**0.331** ± 0.001	**0.209** ± 0.001	**0.205** ± 0.001	**0.192** ± 0.001	0.018 ± 0.001	0.020 ± 0.001	0.019 ± 0.001
Gains	0.014	0.012	0.012	0.046	0.082	0.126	0.002	0.008	0.031	0.008	0.129	0.004

Table 3 The RAE values of various methods on air quality, industry, exchange rate and weather datasets when horizon $\tau \in \{3, 6, 12\}$

Methods	Air quality			Industry			Exchange rate			Weather		
	$\tau = 3$	$\tau = 6$	$\tau = 12$	$\tau = 3$	$\tau = 6$	$\tau = 12$	$\tau = 3$	$\tau = 6$	$\tau = 12$	$\tau = 3$	$\tau = 6$	$\tau = 12$
VAR	0.235 ± 0.001	0.257 ± 0.001	0.242 ± 0.001	0.715 ± 0.001	0.724 ± 0.001	0.509 ± 0.001	0.298 ± 0.001	0.289 ± 0.001	0.304 ± 0.001	0.050 ± 0.001	0.054 ± 0.001	0.057 ±0.001
+Ours	**0.229**± 0.001	**0.251**± 0.001	**0.233**± 0.001	**0.442**± 0.001	**0.454**± 0.001	**0.479**± 0.001	**0.287**± 0.001	**0.229**± 0.001	**0.271**± 0.001	**0.047**± 0.001	**0.050**± 0.001	**0.048**± 0.001
$\Delta(\downarrow)$	0.006	0.006	0.009	0.273	0.270	0.030	0.011	0.060	0.033	0.003	0.004	0.009
VARMA	0.244 ± 0.001	0.272 ± 0.001	0.273 ± 0.001	0.693 ± 0.001	0.691 ± 0.001	0.687 ± 0.001	0.375 ± 0.001	0.371 ± 0.001	0.355 ± 0.001	0.051 ± 0.001	0.055 ± 0.001	0.057 ± 0.001
+Ours	**0.237**± 0.001	**0.265**± 0.001	**0.259**± 0.001	**0.677**± 0.001	**0.669**± 0.001	**0.666**± 0.001	**0.354**± 0.001	**0.354**± 0.001	**0.277**± 0.001	**0.034**± 0.001	**0.039**± 0.001	**0.047**± 0.001
$\Delta(\downarrow)$	0.007	0.007	0.014	0.016	0.022	0.021	0.021	0.017	0.078	0.017	0.016	0.010
CNN	0.346 ± 0.001	0.366 ± 0.001	0.349 ± 0.001	0.432 ± 0.001	0.431 ± 0.001	0.426 ± 0.001	0.306 ± 0.001	0.309 ± 0.001	0.313 ± 0.001	0.147 ± 0.001	0.147 ± 0.001	0.143 ± 0.001
+Ours	**0.291**± 0.012	**0.297**± 0.002	**0.316**± 0.002	**0.335**± 0.015	**0.292**± 0.003	**0.319**± 0.023	**0.251**± 0.001	**0.253**± 0.001	**0.258**± 0.001	0.067 ± 0.029	0.068 ± 0.027	0.068 ± 0.025
$\Delta(\downarrow)$	0.055	0.069	0.033	0.097	0.139	0.107	0.055	0.056	0.055	0.080	0.079	0.075
RNN	0.226 ± 0.001	0.272 ± 0.000	0.251 ± 0.001	0.635 ± 0.001	0.643 ± 0.001	0.678 ± 0.001	0.257 ± 0.000	0.257 ± 0.000	0.257 ± 0.000	0.184 ± 0.136	0.184 ± 0.135	0.198 ± 0.138
+Ours	**0.219**± 0.001	**0.261**± 0.000	**0.242**± 0.001	**0.462**± 0.001	**0.432**± 0.001	**0.371**± 0.001	**0.119**± 0.001	**0.122**± 0.000	**0.126**± 0.001	0.170 ± 0.015	0.167 ± 0.016	0.170 ± 0.017
$\Delta(\downarrow)$	0.007	0.011	0.009	0.173	0.211	0.307	0.138	0.135	0.131	0.014	0.017	0.028
LSTNet	0.237 ± 0.003	0.265 ± 0.002	0.262 ± 0.003	0.449 ± 0.001	0.455 ± 0.001	0.483 ± 0.001	0.293 ± 0.001	0.289 ± 0.001	0.295 ± 0.001	0.237 ± 0.036	0.236 ± 0.040	0.235 ± 0.039
+Ours	**0.222**± 0.004	**0.248**± 0.001	**0.245**± 0.001	**0.301**± 0.008	**0.317**± 0.003	**0.346**± 0.004	**0.121**± 0.001	± 0.001**0.141**	**0.181**± 0.001	0.045 ± 0.001	0.044 ± 0.010	0.045 ± 0.010
$\Delta(\downarrow)$	0.015	0.017	0.017	0.148	0.138	0.137	0.172	0.148	0.114	0.192	0.192	0.190
Timemixer	0.317 ± 0.002	0.351 ± 0.002	0.342 ± 0.001	0.452 ± 0.001	0.443 ± 0.001	0.432 ± 0.001	0.281 ± 0.001	0.294 ± 0.001	0.289 ± 0.001	0.096 ± 0.025	0.129 ± 0.026	0.092 ± 0.016
+Ours	**0.274**± 0.004	**0.303**± 0.007	**0.304**± 0.005	**0.265**± 0.001	**0.279**± 0.004	**0.298**± 0.011	**0.272**± 0.001	**0.281**± 0.001	**0.278**± 0.000	0.073 ± 0.001	0.065 ± 0.001	0.046 ± 0.001

Table 3 (continued)

Methods	Air quality			Industry			Exchange rate			Weather		
	$\tau = 3$	$\tau = 6$	$\tau = 12$	$\tau = 3$	$\tau = 6$	$\tau = 12$	$\tau = 3$	$\tau = 6$	$\tau = 12$	$\tau = 3$	$\tau = 6$	$\tau = 12$
$\Delta(\downarrow)$	0.043	0.048	0.038	0.187	0.164	0.134	0.009	0.013	0.011	0.023	0.064	0.046
Transformer	0.236 ± 0.033	0.296 ± 0.001	0.250 ± 0.001	0.388 ± 0.001	0.372 ± 0.001	0.317 ± 0.001	0.209 ± 0.001	0.210 ± 0.001	0.223 ± 0.001	0.013 ± 0.001	0.074 ± 0.031	0.013 ± 0.031
+Ours	**0.221**± 0.001	**0.272**± 0.001	**0.244**± 0.001	**0.311**± 0.001	**0.353**± 0.001	**0.307**± 0.001	**0.198**± 0.001	**0.193**± 0.001	**0.181**± 0.001	0.009 ± 0.001	0.012 ± 0.001	0.010 ± 0.001
$\Delta(\downarrow)$	0.015	0.024	0.006	0.077	0.019	0.010	0.011	0.017	0.042	0.004	0.062	0.003

Table 4 The CORR values of various methods on air quality, industry, exchange rate and weather datasets when horizon $\tau \in \{3, 6, 12\}$

Methods	Air quality			Industry			Exchange rate			Weather		
	$\tau = 3$	$\tau = 6$	$\tau = 12$	$\tau = 3$	$\tau = 6$	$\tau = 12$	$\tau = 3$	$\tau = 6$	$\tau = 12$	$\tau = 3$	$\tau = 6$	$\tau = 12$
VAR	0.572 ± 0.001	0.460 ± 0.001	0.546 ± 0.001	0.650 ± 0.001	0.657 ± 0.001	0.657 ± 0.001	0.839 ± 0.001	0.672 ± 0.001	0.578 ± 0.001	0.388 ± 0.001	0.379 ± 0.001	0.413 ± 0.001
+Ours	**0.580**± 0.001	**0.472**± 0.001	**0.551**± 0.001	**0.871**± 0.001	**0.860**± 0.001	**0.662**± 0.001	**0.841**± 0.001	**0.833**± 0.001	**0.827**± 0.001	**0.414**± 0.001	**0.387**± 0.001	**0.427**± 0.001
$\Delta(\uparrow)$	0.008	0.012	0.005	0.221	0.203	0.005	0.002	0.161	0.249	0.026	0.008	0.014
VARMA	0.507 ± 0.001	0.394 ± 0.001	0.362 ± 0.001	0.605 ± 0.001	0.580 ± 0.001	0.538 ± 0.001	0.825 ± 0.001	0.821 ± 0.001	0.738 ± 0.001	0.456 ± 0.001	0.423 ± 0.001	0.357 ± 0.001
+Ours	**0.532**± 0.001	**0.403**± 0.001	**0.397**± 0.001	**0.616**± 0.001	**0.594**± 0.001	**0.559**± 0.001	**0.848**± 0.001	**0.844**± 0.001	**0.814**± 0.001	**0.514**± 0.001	**0.523**± 0.001	**0.393**± 0.001
$\Delta(\uparrow)$	0.025	0.009	0.035	0.011	0.014	0.021	0.023	0.023	0.076	0.058	0.100	0.036
CNN	0.191 ± 0.001	0.271 ± 0.002	0.271 ± 0.004	0.849 ± 0.001	0.827 ± 0.001	0.792 ± 0.001	0.834 ± 0.002	0.829 ± 0.001	0.818 ± 0.001	0.712 ± 0.121	0.547 ± 0.101	0.391 ± 0.129
+Ours	**0.469**± 0.061	**0.439**± 0.019	**0.299**± 0.008	**0.885**± 0.005	**0.901**± 0.008	**0.798**± 0.005	**0.906**± 0.001	**0.902**± 0.001	**0.893**± 0.001	0.746 ± 0.001	0.615 ± 0.017	0.460 ± 0.033
$\Delta(\uparrow)$	0.278	0.168	0.028	0.036	0.074	0.006	0.072	0.073	0.075	0.034	0.068	0.069
RNN	0.604 ± 0.001	0.481 ± 0.001	0.517 ± 0.001	0.754 ± 0.001	0.742 ± 0.001	0.723 ± 0.001	0.872 ± 0.001	0.869 ± 0.001	0.863 ± 0.001	0.707 ± 0.355	0.545 ± 0.277	0.121 ± 0.159
+Ours	**0.649**± 0.001	**0.587**± 0.001	**0.543**± 0.001	**0.836**± 0.001	**0.843**± 0.001	**0.834**± 0.001	**0.931**± 0.001	**0.922**± 0.001	**0.905**± 0.001	0.712 ± 0.114	0.630 ± 0.116	0.446 ± 0.042
$\Delta(\uparrow)$	0.045	0.106	0.026	0.082	0.101	0.111	0.059	0.053	0.042	0.005	0.085	0.325
LSTNet	0.567 ± 0.002	0.502 ± 0.008	0.496 ± 0.002	0.784 ± 0.001	0.769 ± 0.001	0.744 ± 0.001	0.809 ± 0.001	0.805 ± 0.001	0.796 ± 0.001	0.103 ± 0.026	0.110 ± 0.091	0.126 ± 0.098
+Ours	**0.647**± 0.005	**0.623**± 0.003	**0.562**± 0.003	**0.816**± 0.019	**0.785**± 0.005	**0.757**± 0.005	**0.945**± 0.001	**0.932**± 0.001	**0.909**± 0.001	0.164 ± 0.007	0.176 ± 0.007	0.213 ± 0.001
$\Delta(\uparrow)$	0.080	0.121	0.066	0.032	0.016	0.013	0.136	0.127	0.113	0.061	0.066	0.087
Timemixer	0.201 ± 0.012	0.292 ± 0.015	0.261 ± 0.003	0.803 ± 0.001	0.788 ± 0.001	0.761 ± 0.001	0.786 ± 0.001	0.779 ± 0.001	0.771 ± 0.001	0.602 ± 0.046	0.451 ± 0.025	0.234 ± 0.047
+Ours	**0.478**± 0.044	**0.441**± 0.009	**0.304**± 0.009	**0.907**± 0.006	**0.891**± 0.004	**0.823**± 0.033	**0.867**± 0.001	**0.856**± 0.001	**0.853**± 0.001	0.672 ± 0.029	0.537 ± 0.008	0.404 ± 0.038

Table 4 (continued)

Methods	Air quality			Industry			Exchange rate			Weather		
	$\tau = 3$	$\tau = 6$	$\tau = 12$	$\tau = 3$	$\tau = 6$	$\tau = 12$	$\tau = 3$	$\tau = 6$	$\tau = 12$	$\tau = 3$	$\tau = 6$	$\tau = 12$
$\Delta(\uparrow)$	0.277	0.149	0.043	0.104	0.103	0.062	0.081	0.077	0.082	0.070	0.086	0.170
Transformer	0.610 ± 0.024	0.497 ± 0.001	0.407 ± 0.001	0.914 ± 0.001	0.897 ± 0.001	0.871 ± 0.001	0.677 ± 0.001	0.675 ± 0.001	0.626 ± 0.001	0.708 ± 0.033	0.455 ± 0.029	0.270 ± 0.005
+Ours	**0.628**± 0.001	**0.564**± 0.001	**0.421**± 0.001	**0.921**± 0.001	**0.902**± 0.001	**0.886**± 0.001	**0.885**± 0.001	**0.898**± 0.001	**0.835**± 0.001	0.744 ± 0.030	0.556 ± 0.043	0.498 ± 0.018
$\Delta(\uparrow)$	0.018	0.067	0.014	0.007	0.005	0.015	0.208	0.223	0.209	0.036	0.101	0.228

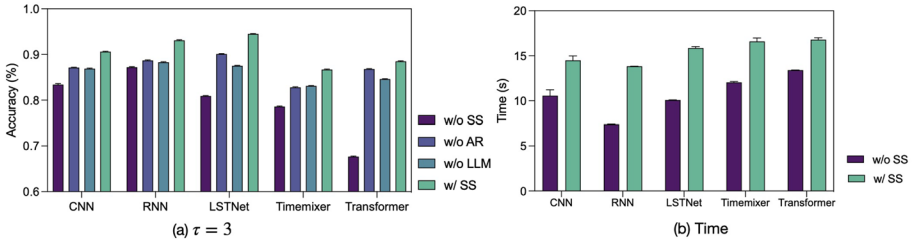


Fig. 6 Ablation studies of model performance in subfigure (a) and extra computational costs in subfigure (b)

- *w/o SS*: The model in Fig. 2 without SS module.
- *Random-M*: The SS module uses a randomly initialized M without gradient updating, as shown in Fig. 2.
- *Linear-M*: The SS module uses a fixed linear combination between X and $\hat{X}$, as shown in Fig. 2.
- *w/SS*: The model in Fig. 2 with the LLM-guided SS module and AR component.

The empirical results in Table 5 on Exchange-rate dataset show that our proposed *w/SS* consistently outperforms both Random-M and Linear-M, due to that the LLM-guided SS module can leverage domain knowledge to adaptively integrate trend information for better generalization in real-world scenarios. The exchange-rate dataset exhibits a long-term trend patterns and overall strong predictive performance. As the prediction horizon increases, the influence of these short-term fluctuations diminishes, and the long-term trend becomes dominant for predictive performance. The key advantage of SS module lies in adaptively utilizing trend information. By effectively capturing these trends, it achieves strong performance in long-term predictions compared to short-term settings, which effectively reverses the performance degradation typically observed in other competitive baselines.

Then we evaluate the additional computational costs of variants including naive interpolation like *Linear-M* and *Random-M* in Table 6. The empirical results show that there is minimal difference in time between these methods, due to that we interact with the LLM exclusively through API calls (without fine-tuning) to provide external supervisory signals during training and generate natural language explanations during interpretation based on the SS module. Thus, the network latency associated with API calls is the dominant computational cost. The LLM-guided SS module via API calls does not introduce significant additional computational costs, making it particularly suitable for multivariate time series forecasting applications with limited computational resources.

Irregularly Sampled Dataset

Based on above observations, we find that integrating LLMs may introduce a potential point of failure, thereby necessitating a fail-safe strategy. In the ablation study, we implement a smoothness-based sanity check strategy. Specifically, we utilize the second-order difference of the generated mask to detect anomalies. If the fluctuations exceed a pre-defined threshold, the SS module attempts to regenerate the augmented sample. If repeated attempts fail to satisfy the smoothness constraint, the sample is rejected to avoid introducing anomalous patterns into the training process. Following the settings in (Cao et al., 2018; Chen et al., 2022; Du et al., 2023), we simulate non-uniform sampling by randomly masking 10%

Table 5 The performance of variants including naive interpolations on exchange-rate datasets with$\tau \in \{3, 6, 12\}$

Setting	RSE			RAE			CORR		
	$\tau = 3$	$\tau = 6$	$\tau = 12$	$\tau = 3$	$\tau = 6$	$\tau = 12$	$\tau = 3$	$\tau = 6$	$\tau = 12$
w/o SS	0.211 ± 0.001	0.213 ± 0.001	0.223 ± 0.01	0.209 ± 0.001	0.210 ± 0.001	0.223 ± 0.001	0.677 ± 0.001	0.675 ± 0.001	0.626 ± 0.001
Random-M	0.252 ± 0.001	0.271 ± 0.001	0.276 ± 0.001	0.269 ± 0.001	0.281 ± 0.001	0.296 ± 0.001	0.714 ± 0.001	0.823 ± 0.001	0.766 ± 0.001
Linear-M	0.268 ± 0.001	0.276 ± 0.001	0.272 ± 0.001	0.281 ± 0.001	0.295 ± 0.001	0.283 ± 0.001	0.829 ± 0.001	0.818 ± 0.001	0.752 ± 0.001
w/SS	0.209 ± 0.003	0.205 ± 0.002	0.192 ± 0.004	0.198 ± 0.003	0.193 ± 0.006	0.181 ± 0.002	0.885 ± 0.002	0.898 ± 0.003	0.835 ± 0.003

Table 6 The extra computational costs of variants

Method	w/o SS	Random-M	Linear-M	w/SS
CNN	10.567 ± 0.649	10.911 ± 0.296	11.231 ± 0.216	14.491 ± 0.496
RNN	7.404 ± 0.009	7.941 ± 0.003	8.121 ± 0.004	13.828 ± 0.014
LSTNet	10.086 ± 0.010	11.641 ± 0.010	10.981 ± 0.012	15.861 ± 0.146
Timemixer	12.042 ± 0.085	12.491 ± 0.085	13.111 ± 0.083	16.598 ± 0.371
Transformer	13.408 ± 0.013	14.212 ± 0.013	13.982 ± 0.014	16.789 ± 0.213

and 30% time steps from the original dataset, representing varying levels of irregularly sampled dataset. As shown in Table 7, *"w/Check"* denotes a variant that applies a sanity-check strategy when the LLM-guided SS module generates augmented samples, while *"w/o Check"* corresponds to the original method without the strategy. The empirical results show that the LLM-guided SS module consistently improves predictive performance on irregularly sampled datasets, and the fail-safe strategy improves predictive performance in most settings, as SS module facilitates deep models in capturing more robust representations.

4.3 Experimental Results on Interpretation

To verify that our interpretation method based on the SS module can perceive key trend information, we conduct empirical results from qualitative and quantitative perspectives in Sect. 4.3.1 and Sect. 4.3.2.

4.3.1 Qualitative Analysis

We apply the SS module to the pre-trained Transformer on the Air Quality dataset. Figure 7 visualizes the feature importance map for future time steps with horizon $\tau = 6$, where a lighter color indicates higher feature importance. Figure 7 (a) shows the feature importance map, and Fig. 7 (b) shows NO_2 and its frequency domain representation via Fast Fourier Transform (FFT) (Nussbaumer & Nussbaumer, 1982). Figure 7 (c) shows each feature importance, whereas Fig. 7 (d) provides a boxplot that shows the temporal distribution of the feature importance on Air Quality dataset. These observations show that the fluctuations in the time interval [5,6] critically affect the predictions. Note that NO_2 shows a strong correlation with noxious gases and airborne chemicals, which aligns with existing knowledge that higher NO_2 may be associated with more severe pollution (Costa et al., 2014). This qualitative analysis validates the importance of capturing implicit correlations and demonstrates the effectiveness of the SS module.

Additionally, the interpretation method based on SS module can not only provide intuitive saliency analysis, but also leverage LLM's domain knowledge to generate natural language explanations that offer deep insights like Fig. 8. Consequently, non-expert researchers can make preliminary assessments of air quality, like "The rise in NO_2 may indicate severe air pollution", while domain experts can observe other noxious gases (NO_x and O_3) through an in-depth analysis and precisely pinpoint pollution sources. The interpretation method enhances confidence in the application of artificial intelligence to multivariate time series forecasting.

Table 7 The performance comparison on irregularly sampled time series datasets including Aotizhongxin and Changping under large language model and fail-safe guidance. The best performance is highlighted in boldface

Dataset	Ratio	Model	RSE			RAE			CORR		
			3	6	12	3	6	12	3	6	12
Aotizhongxin	10%	LSTNet	0.306 ± 0.002	0.373 ± 0.001	0.480 ± 0.001	0.134 ± 0.001	0.165 ± 0.001	0.211 ± 0.001	0.614 ± 0.001	0.557 ± 0.002	0.550 ± 0.001
		w/o Check	0.290 ± 0.004	0.360 ± 0.003	0.476 ± 0.002	0.122 ± 0.002	0.155 ± 0.002	0.202 ± 0.002	0.709 ± 0.004	0.692 ± 0.002	0.601 ± 0.002
		w/Check	**0.285**± 0.001	**0.351**± 0.002	**0.451**± 0.001	**0.118**± 0.001	**0.152**± 0.001	**0.196**± 0.004	**0.780**± 0.002	**0.715**± 0.003	**0.639**± 0.001
	30%	LSTNet	0.334 ± 0.003	0.506 ± 0.002	0.540 ± 0.002	0.138 ± 0.002	0.207 ± 0.001	0.240 ± 0.001	0.719 ± 0.001	0.535 ± 0.002	0.426 ± 0.001
		w/o Check	0.325 ± 0.002	0.420 ± 0.001	0.517 ± 0.001	**0.128**± 0.001	0.177 ± 0.002	0.221 ± 0.003	0.732 ± 0.003	0.685 ± 0.004	0.586 ± 0.002
		w/Check	**0.322**± 0.004	**0.408**± 0.001	**0.507**± 0.002	0.129± 0.001	**0.167**± 0.001	**0.214**± 0.002	**0.745**± 0.001	**0.709**± 0.002	**0.632**± 0.001
Changping	10%	LSTNet	0.351± 0.001	0.380± 0.002	0.489± 0.002	0.138± 0.001	0.158± 0.002	0.201± 0.001	0.742± 0.003	0.581± 0.001	0.546± 0.002
		w/o Check	0.341± 0.004	0.365± 0.003	0.473± 0.001	**0.129**± 0.002	0.147± 0.001	0.190± 0.003	0.773± 0.001	0.693± 0.003	0.572± 0.002
		w/Check	**0.339**± 0.001	**0.354**± 0.002	**0.463**± 0.001	0.132± 0.001	**0.142**± 0.001	**0.186**± 0.002	**0.790**± 0.001	**0.741**± 0.002	**0.599**± 0.001
	30%	LSTNet	0.403± 0.003	0.413 ± 0.001	0.507± 0.001	0.144± 0.001	0.159± 0.001	0.195± 0.002	0.743± 0.002	0.582± 0.003	0.570± 0.001
		w/o Check	0.398 ± 0.001	0.405 ± 0.002	0.491± 0.001	0.141 ± 0.001	0.151± 0.002	0.192± 0.001	0.758± 0.001	0.699± 0.002	0.595± 0.001
		w/Check	**0.394**± 0.001	**0.397**± 0.001	**0.483**± 0.002	**0.137**± 0.002	**0.147**± 0.001	**0.188**± 0.001	**0.784**± 0.002	**0.727**± 0.001	**0.614**± 0.002

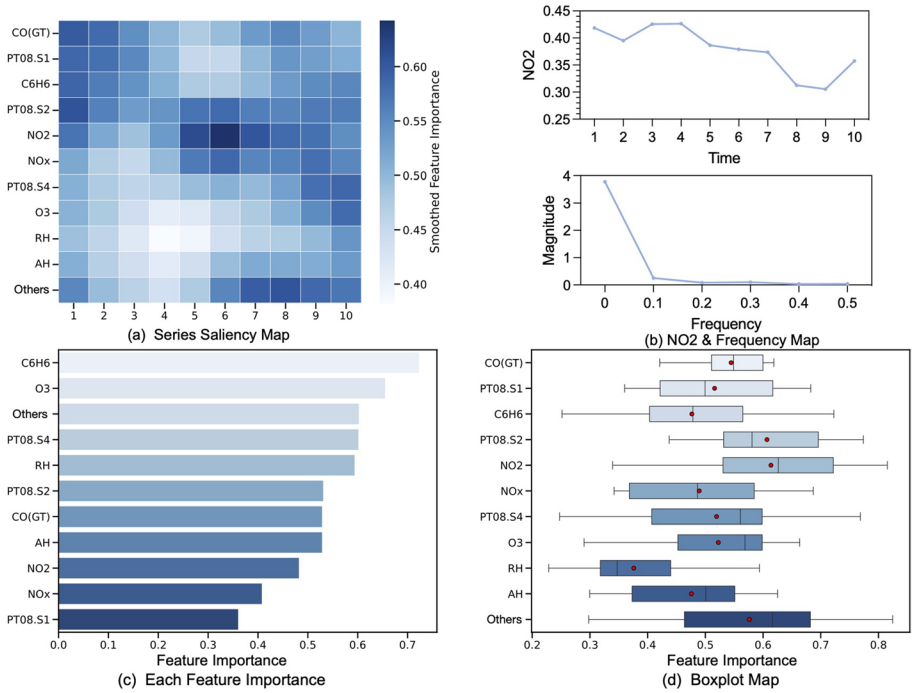


Fig. 7 The visualization of feature importance map for transformer based LLM-guided SS module with horizon $\tau = 3$ on Air Quality dataset. Subfigure (a) shows the Series Saliency values, Subfigure (b) shows the NO_2 with its corresponding frequency map, Subfigure (c) shows the each feature importance, and Subfigure (d) shows the boxplot of feature importance

> Feature importance highlights the key variables for the model's predictions. Feature 6 is the most important. Features 0 and 5 also play significant roles, likely reflecting key trends or cyclical patterns. Features 2 and 7 are moderately important, indicating they provide additional, though less dominant signals. Features 3, 4, and 1 have smaller effects, potentially representing secondary interactions or lagged influences. Feature 8 has relatively low impact, while Features 9 and 10 appear to be noisy and contribute less to the model's predictions.

Fig. 8 The SS module generates natural language explanations by leveraging the LLM's knowledge to provide deep insights

4.3.2 Quantitative Analysis

To comprehensively evaluate the effectiveness of interpretability, we propose a quantitative metric called *Area Under CORR curve* (AUCORR). Specifically, for each sample X in the test set, we employ a given interpretation method to generate the corresponding feature impor-

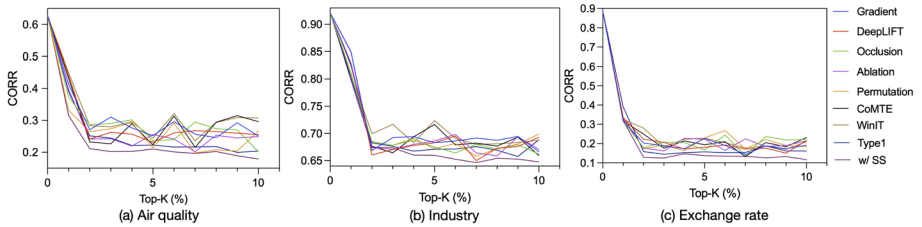


Fig. 9 The decreasing CORR curves with self-attention encoder for air quality, industry and exchange rate datasets with horizon $\tau = 3$

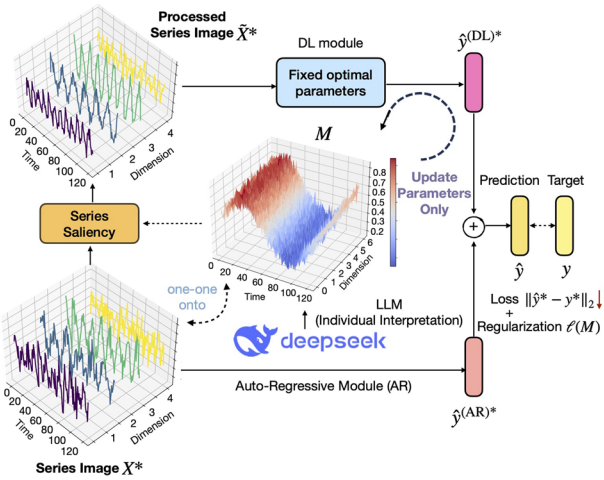


Fig. 4 The interpretation process based on the Series Saliency module. We only update the SS module to identify trend information in feature importance map for prediction and leverage the LLM's domain knowledge to provide intuitive explanations. Best viewed in color

tance map $F(X)$ and select the top $K\%$ features ranked by importance score, where $F_e(X_{t,i})$ is the e-th element in $\{F_e(X_{t,i})\}_{e=1}^{T\times D}$. The top $M\%$ features that satisfy $\sum_{i=1}^{T}\sum_{t=1}^{D} F_e(X_{t,i}) \approx M\%$ are then perturbed by Gaussian noise $\mathcal{N}(\mu, \sigma^2)$, where $\mu = 0$ and $\sigma^2 = 0.1$. Then, we adjust the percentage $[1\%, 2\%, \ldots, 10\%]$ to generate the performance degradation curve and compute its area under the curve as AUCORR. A steep performance decline indicates that the perturbed features are critical for accurate predictions, whereas minimal changes in performance suggest that the perturbed features have little impact. Consequently, a lower AUCORR (i.e., a smaller area under the curve) implies that the corresponding interpretation methods effectively capture critical features, such as trend information, for more accurate predictions.

We compare the interpretation methods based on the SS module with other competitive baselines, including Gradient (Baehrens et al., 2010), DeepLift (Shrikumar & Greenside, 2017), Occlusion (Zeiler, 2014), Ablation (Suresh et al., 2017), Permutation (Molnar, 2020), CoMTE (Ates et al., 2021), WinIT (Leung et al., 2023).

To further examine the effect of the regularizer ℓ_m in Eq. (11), we consider the following variants of our method as

- *Type 1*: The model in Fig. 4 is interpreted without regularizer ℓ_m in Eq. (11).

Fig. 5 The prompt P provided to the LLM in the interpretation process

> Below are the feature importance scores M^* obtained from a pretrained deep learning model. Please interpret these feature importance scores M^* like why these features are critical to predictive performance.

Table 8 The AUCORR values of various interpretable methods on air quality, industry, and exchange rate datasets with horizon$\tau = 3$. Best performance in boldface

Dataset	Air quality	Industry	Exchange rate
Gradient (Baehrens et al., 2010)	0.03052	0.07141	0.02342
DeepLIFT (Shrikumar & Greenside, 2017)	0.02928	0.07043	0.02356
Occlusion (Zeiler, 2014)	0.02981	0.07203	0.02644
Ablation (Suresh et al., 2017)	0.02735	0.07051	0.02431
Permutation (Molnar, 2020)	0.02725	0.07039	0.02482
CoMTE (Ates et al., 2021)	0.03003	0.07098	0.02404
WinIT (Leung et al., 2023)	0.03179	0.07171	0.02527
Type1	0.02596	0.06979	0.02182
w/SS	**0.02333**	**0.06857**	**0.01885**

- *w/SS*: The model in Fig. 4 is interpreted with both SS and regularizer ℓ_m in Eq. (11).

From Table 8 and Fig. 9, we can observe that the proposed method outperforms other competitive baselines in all settings. Moreover, the regularization term ℓ_m further enhances interpretability. Although it is difficult to guarantee that any selected future values will have the representative interpretation in qualitative analysis like Fig. 7, the performance of our method achieves a superior performance compared to other competitive interpretation baselines in Table 8.

5 Conclusion

In this work, we propose the Series Saliency (SS) module, and theoretically demonstrate that it can perceive trend information while exploring the previously uncharted sample space, to improve generalization and interpretability in multivariate time series forecasting. We extract "series image" by sliding the window to coherently consider both the time and the feature dimensions. In the training process, we use the LLM's domain knowledge to guide the SS module in adaptively integrating trend information to improve accuracy. While in the interpretation process, we update the SS module to identify key trend patterns for prediction and use LLM's domain knowledge to provide natural language explanations. We also design a theoretically guaranteed smoothing regularization term to avoid anomalous patterns. Extensive empirical results from both qualitative and quantitative perspectives show that the SS module significantly improves the performance of various deep models in a plug-in manner. Ablation studies further verify the significance of each component in the SS module. Future work will explore how trend-aware learning and LLM-guided interpretation can further improve multivariate time series forecasting.

Appendix

A Summary of Changes

The paper is a substantial extension of the preliminary conference paper titled "*Two Birds with One Stone: Series Saliency for Accurate and Interpretable Multivariate Time Series Forecasting*" published at International Joint Conference on Artificial Intelligence (IJCAI) (Pan & Hu, 2021). A brief summary of the major extensions is as follows:

- The current version combines the Large Language Model (LLM)'s knowledge (Liu et al., 2024) with the statistical features of time series to guide SS module in integrating trend information with the original data to improve accuracy (see Sects. 3.2,3.3 and 4.2), while the conference version focused only on the blind combination of trend information that potentially disregards trend patterns;
- The current version leverages LLM's domain knowledge to perceive key patterns in the interpretation process and provides natural language interpretations for domain experts (see Sects. 3.4 and 4.3), while the conference version considered only a particular gradient-based implementation;
- The current version theoretically demonstrates that the SS module can effectively explore sample space while preserving the key trend information from the convex optimization perspective (see Sect. 3.2.1 Theorem 1 and Appendix B.1);
- The current version also theoretically demonstrates that the subtle regularization term guides SS module to avoid introducing sawtooth like anomalous patterns, while satisfying the theoretical upper bounds (see Sect. 3.4.2 Theorem 2 and Appendix B.2);
- The current version conducts more extensive experiments from both qualitative and quantitative perspectives on various state-of-art deep architectures (e.g., Transformer (Vaswani et al., 2017), TimeMixer (Wang et al., 2024)). The results demonstrate that the current version significantly outperforms the conference version (see Sect. 4.2.2) and improves the performance of various deep models as a plug-in manner (see Sect. 4.2 and Sect. 4.3);
- The current version is significantly re-written with more related work discussed and more insights provided throughout the paper.

B Proof

B.1 Proof of Theorem 1

Proof **Part I: Convexity.**

From the definition, any processed value $\tilde{x}_{t,i}$ is formed by taking a convex combination of the original sample $x_{t,i}$ and the smoothed value $\hat{x}_{t,i}$. In other words,

$$\tilde{x}_{t,i} = \alpha_{t,i}\,\hat{x}_{t,i} \;+\; \big(1-\alpha_{t,i}\big)\,x_{t,i}, \quad \alpha_{t,i} \in [0,1], \tag{18}$$

where $\alpha_{t,i}$ can vary across (t, i), which means that for each time step t and feature i, the point $\tilde{x}_{t,i}$ lies on the line between $x_{t,i}$ and $\hat{x}_{t,i}$.

Define the set

$$\mathcal{C} \;=\; \Big\{ \tilde{X} \in \mathbb{R}^{T\times D} \;\big|\; \tilde{x}_{t,i} = \alpha_{t,i}\,\hat{x}_{t,i} + \big(1-\alpha_{t,i}\big)\,x_{t,i},\ \forall\,(t,i),\ \alpha_{t,i} \in [0,1] \Big\}. \tag{19}$$

To show that $\mathcal{C}$ is convex in $\mathbb{R}^{T\times D}$, we consider any two points

$$\tilde{x}^{(1)}_{t,i},\ \tilde{x}^{(2)}_{t,i} \;\in\; \mathcal{C}. \tag{20}$$

By definition, there exist $\alpha^{(1)}_{t,i}, \alpha^{(2)}_{t,i} \in [0,1]$ such that

$$\begin{aligned} \tilde{x}^{(1)}_{t,i} &= \alpha^{(1)}_{t,i}\,\hat{x}_{t,i} + \big(1-\alpha^{(1)}_{t,i}\big)\,x_{t,i}, \\ \tilde{x}^{(2)}_{t,i} &= \alpha^{(2)}_{t,i}\,\hat{x}_{t,i} + \big(1-\alpha^{(2)}_{t,i}\big)\,x_{t,i}. \end{aligned} \tag{21}$$

Take a convex combination of $\tilde{x}^{(1)}_{t,i}$ and $\tilde{x}^{(2)}_{t,i}$ with $\lambda \in [0,1]$:

$$\lambda\,\tilde{x}^{(1)}_{t,i} \;+\; (1-\lambda)\,\tilde{x}^{(2)}_{t,i} = \lambda\big(\alpha^{(1)}_{t,i}\,\hat{x}_{t,i} + \big(1-\alpha^{(1)}_{t,i}\big)\,x_{t,i}\big) \;+\; (1-\lambda)\big(\alpha^{(2)}_{t,i}\,\hat{x}_{t,i} + \big(1-\alpha^{(2)}_{t,i}\big)\,x_{t,i}\big). \tag{22}$$

Rearrange terms to get

$$\big[\,\lambda\,\alpha^{(1)}_{t,i} + (1-\lambda)\,\alpha^{(2)}_{t,i}\big]\hat{x}_{t,i} \;+\; \Big[1 - \big(\lambda\,\alpha^{(1)}_{t,i} + (1-\lambda)\,\alpha^{(2)}_{t,i}\big)\Big]x_{t,i}. \tag{23}$$

Since

$$\lambda\,\alpha^{(1)}_{t,i} + (1-\lambda)\,\alpha^{(2)}_{t,i} \;\in\; [0,1], \tag{24}$$

this combination is again a valid convex combination of $x_{t,i}$ and $\hat{x}_{t,i}$. Hence, the resulting point lies in $\mathcal{C}$, establishing that $\mathcal{C}$ is convex in each coordinate. Because we are taking the product of these intervals across all (t, i), the overall set remains convex in higher dimensions.

Part II: Range-Boundedness, i.e., $\min(\hat{x}_{t,i}, x_{t,i}) \le \tilde{x}_{t,i} \le \max(\hat{x}_{t,i}, x_{t,i})$.

We observe that for each (t, i),

$$\tilde{x}_{t,i} = \alpha_{t,i}\,\hat{x}_{t,i} + \big(1-\alpha_{t,i}\big)\,x_{t,i}, \tag{25}$$

with $\alpha_{t,i} \in [0,1]$. Consequently,

$$\tilde{x}_{t,i} \;\in\; \Big[\min(\hat{x}_{t,i}, x_{t,i}),\ \max(\hat{x}_{t,i}, x_{t,i})\Big]. \tag{26}$$

Hence, each coordinate of the mixed series image lie between $\hat{x}_{t,i}$ and $x_{t,i}$. So the Cartesian product

$$\bigotimes_{t,i}\Big[\min(\hat{x}_{t,i}, x_{t,i}),\ \max(\hat{x}_{t,i}, x_{t,i})\Big] \tag{27}$$

is contained within the convex hull of $x_{t,i}$ and $\hat{x}_{t,i}$ for all (t, i). □

B.2 Proof of Theorem 2

Proof By definition in Eq. (4), we have

$$\tilde{x}_{t,i} = m_{t,i}\,\hat{x}_{t,i} \;+\; \big(1 - m_{t,i}\big)\,x_{t,i}. \tag{28}$$

Hence, using the triangle inequality on the difference,

$$\begin{aligned}
\tilde{x}_{t+1,i} - \tilde{x}_{t,i} &= \big[\, m_{t+1,i}\,\hat{x}_{t+1,i} + \big(1 - m_{t+1,i}\big)\,x_{t+1,i}\big] \;-\; \big[\, m_{t,i}\,\hat{x}_{t,i} + \big(1 - m_{t,i}\big)\,x_{t,i}\big] \\
&= m_{t+1,i}\,\big(\hat{x}_{t+1,i} - x_{t+1,i}\big) \;+\; m_{t,i}\,\big(x_{t,i} - \hat{x}_{t,i}\big) \\
&\quad +\; m_{t,i}\,\hat{x}_{t+1,i} \;-\; m_{t,i}\,x_{t+1,i} \;+\; \big(x_{t+1,i} - x_{t,i}\big) \\
&= \big(m_{t+1,i} - m_{t,i}\big)\big(\hat{x}_{t+1,i} - x_{t+1,i}\big) + m_{t,i}\,\big(\hat{x}_{t+1,i} - \hat{x}_{t,i}\big) \;+\; \big(x_{t+1,i} - x_{t,i}\big).
\end{aligned}$$

Taking absolute values,

$$\big|\tilde{x}_{t+1,i} - \tilde{x}_{t,i}\big| \;\le\; \big|m_{t+1,i} - m_{t,i}\big|\,\big|\hat{x}_{t+1,i} - x_{t+1,i}\big| \;+\; \big|m_{t,i}\big|\,\big|\hat{x}_{t+1,i} - \hat{x}_{t,i}\big| \;+\; \big|x_{t+1,i} - x_{t,i}\big|. \tag{29}$$

Since $\displaystyle\sum_{t=1}^{T}\sum_{i=1}^{D}\big(m_{t+1,i} - m_{t,i}\big)^2 \;\le\; \varepsilon$, we know that

$$\max_{t,i}\,\big|m_{t+1,i} - m_{t,i}\big| \;\le\; \sqrt{\epsilon}. \tag{30}$$

By definition in Eq. (4), we have $|m_{t+1,i}| \le 1$. Then for each (t, i),

$$\big|m_{t,i}\big| \;=\; \big|m_{t+1,i} + (m_{t,i} - m_{t+1,i})\big| \;\le\; \big|m_{t+1,i}\big| \;+\; \big|m_{t,i} - m_{t+1,i}\big| \;\le\; 1 + \sqrt{\epsilon}. \tag{31}$$

(If $m_{t,i} \in [0, 1]$, then simply $|m_{t,i}| \le 1$ also works for coarse bounding.)

Then we bound $|\hat{x}_{t+1,i} - \hat{x}_{t,i}|$. Recall the definition of Gaussian smoothing:

$$\hat{x}_{t,i} = \sum_{k=-K}^{K} x_{t+k,i}\,g_{\sigma^2}(k), \qquad \sum_{k=-K}^{K} g_{\sigma^2}(k) = 1. \tag{32}$$

Then,

$$\hat{x}_{t+1,i} - \hat{x}_{t,i} \;=\; \sum_{k=-K}^{K}\Big(x_{t+1-k,i} - x_{t-k,i}\Big)\,g_{\sigma^2}(k). \tag{33}$$

Applying the triangle inequality and using $\displaystyle\sum_{k=-K}^{K} g_{\sigma^2}(k) = 1$, we get

$$|\hat{x}_{t+1,i} - \hat{x}_{t,i}| \;\le\; \sum_{k=-K}^{K} |x_{t+1-k,i} - x_{t-k,i}|\, g_{\sigma^2}(k) \;\le\; \max_k |x_{t+1-k,i} - x_{t-k,i}| \;\le\; C_x. \quad (34)$$

Moreover, smoothing typically does not increase the local fluctuation, so

$$|\hat{x}_{t+1,i} - \hat{x}_{t,i}| \;\le\; |x_{t+1,i} - x_{t,i}|, \quad (35)$$

which implies there exists $0 \le \alpha \le 1$ such that

$$|\hat{x}_{t+1,i} - \hat{x}_{t,i}| = \alpha\, |x_{t+1,i} - x_{t,i}| \;\le\; \alpha C_x. \quad (36)$$

Putting these bounds together gives

$$|\tilde{x}_{t+1,i} - \tilde{x}_{t,i}| \;\le\; \sqrt{\epsilon}\,|\hat{x}_{t+1,i} - x_{t+1,i}| \;+\; \left(1+\sqrt{\epsilon}\right)|\hat{x}_{t+1,i} - \hat{x}_{t,i}| \;+\; |x_{t+1,i} - x_{t,i}|. \quad (37)$$

For $|\hat{x}_{t+1,i} - x_{t+1,i}|$, we have

$$|\hat{x}_{t+1,i} - x_{t+1,i}| = \left| \sum_{k=-K}^{K} x_{t+1+k,i} g_{\sigma^2}(k) - x_{t+1,i} \right| \le \sum_{k=-K}^{K} |x_{t+1+k,i} - x_{t+1,i}| g_{\sigma^2}(k). \quad (38)$$

Since $|x_{t+1+k,i} - x_{t+1,i}| \le C_x$, we get

$$|\tilde{x}_{t+1,i} - \tilde{x}_{t,i}| \le \sqrt{\epsilon} \cdot K(K-1)C_x + \alpha(1+\sqrt{\epsilon})C_x + C_x. \quad (39)$$

Finally, this simplifies to:

$$|\tilde{x}_{t+1,i} - \tilde{x}_{t,i}| \le \left[\sqrt{\epsilon} \cdot (K(K-1) + \alpha(\sqrt{\epsilon}+1) + 1\right] C_x. \quad (40)$$

- Remark 1 *(Edge cases)* If $\varepsilon = 0$, then $m_{t+1,i} = m_{t,i}$ for all t, i. The bound reduces to $(\alpha + 1)\, C_x$.

□

C More Experimental Details

C.1 Statistical Properties of Datasets

Table 9 shows the key statistical properties of these datasets, where T denotes the length of time series or data size, D denotes the number of variables, and L is the sampling rate.

Table 9 Statistics of datasets

Datasets	T	D	L
Air quality	9,358	12	1 h
Industry	3,205	12	1 day
Exchange rate	7,588	8	1 h

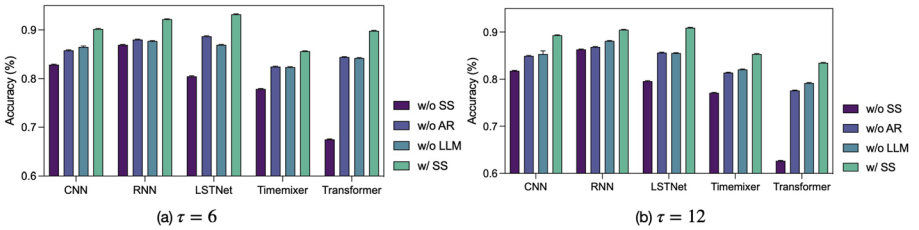


Fig. 10 Ablation study of Transformers on the Air Quality dataset with horizon $\tau = 6$ and $\tau = 12$

C.2 Ablation Study

We visualize the ablation studies of Transformer in air quality with horizons $\tau = 6$ and $\tau = 12$ in Fig. 10 respectively. The results show that both the AR component and the guidance of LLM contribute significantly to performance improvements. Removing either component leads to degenerated performance.

Supplementary Information The online version contains supplementary material available at https://doi.org/10.1007/s10994-026-07045-7.

Author Contributions All authors contributed to the research conception and design. Material preparation, data collection, and analysis were performed by Q. P., L. W., X. C. and N. C. The first version of the manuscript was written by Q. P. and all authors commented on previous versions of the manuscript. All authors read and approved the final manuscript.

Funding This work is supported by the NSF of China Projects (Nos. 62276149, 92370124, 62350080, 92248303, U2341228, 62061136001), BNRist (BNR2022RC01006) and Tsinghua Institute for Guo Qiang; N.C is supported by the Tsinghua High Performance Computing Center and Tsinghua University-Bosch Joint Research Center for Machine-Learning.
L.W. is also supported by the Postdoctoral Fellowship Program of CPSF under Grant Number GZB20230350 and the Shuimu Tsinghua Scholar.

Data Availability No datasets were generated or analysed during the current study.

Code Availability The source code associated with this work is publicly available at: https://github.com/pqy000/seriessaliency_MLJ.

Declarations

Conflict of interest The authors declare no Conflict of interest.

Ethical Approval Not applicable.

References

Ates, E., Aksar, B., Leung, V.J., & Coskun A.K. (2021). Counterfactual explanations for multivariate time series. In: 2021 international conference on applied artificial intelligence (ICAPAI), IEEE, pp 1–8

Baehrens, D., Schroeter, T., Harmeling, S., Kawanabe, M., Hansen, K., & Müller, K.-R. (2010). How to explain individual classification decisions. *Journal of Machine Learning Research, 11*, 1803–1831.

Box, G. E., & Jenkins, G. M. (1976). *Time series analysis, forecasting and control*. Holden-Day.

Cao, W., Wang, D., Li, J., Zhou, H., Li, L., & Li, Y. (2018). Brits: Bidirectional recurrent imputation for time series. Advances in neural information processing systems 31

Castelvecchi, D. (2016). Can we open the black box of AI? *Nature News, 538*(7623), 20.

Chakraborty, S., Tomsett, R., Raghavendra, R., Harborne, D., Alzantot, M., Cerutti, F., Srivastava, M., Preece, A., Julier, S., Rao, R. M., and others (2017). Interpretability of deep learning models: A survey of results. *2017 IEEE smartworld, ubiquitous intelligence & computing, advanced & trusted computed, scalable computing & communications, cloud & big data computing* (pp. 1–6). Internet of people and smart city innovation (smartworld/SCALCOM/UIC/ATC/CBDcom/IOP/SCI): IEEE.

Chaudhari, S., Mithal, V., Polatkan, G., & Ramanath, R. (2021). An attentive survey of attention models. *ACM Transactions on Intelligent Systems and Technology, 12*(5), 1–32.

Che, Z., Purushotham, S., Cho, K., Sontag, D., & Liu, Y. (2018). Recurrent neural networks for multivariate time series with missing values. *Scientific Reports, 8*(1), 1–12.

Chen, Y., Li, Z., Yang, C., Wang, X., Long, G., & Xu, G. (2022). Adaptive graph recurrent network for multivariate time series imputation. In: International Conference on Neural Information Processing, Springer, pp 64–73

Chung, J., Gulcehre, C., Cho, K., & Bengio, Y. (2014). Empirical evaluation of gated recurrent neural networks on sequence modeling. arXiv:1412.3555

Costa, S., Ferreira, J., Silveira, C., Costa, C., Lopes, D., Relvas, H., Borrego, C., Roebeling, P., Miranda, A. I., & Paulo Teixeira, J. (2014). Integrating health on air quality assessment-review report on health risks of two major European outdoor air pollutants: Pm and no2. *Journal of Toxicology and Environmental Health, Part B: Critical Reviews, 17*(6), 307–340.

Cui, P., Hu, W., & Zhu, J. (2020). Calibrated reliable regression using maximum mean discrepancy. *Advances in Neural Information Processing Systems, 33*, 17164–17175.

Dabkowski, P., & Gal, Y. (2017). Real time image saliency for black box classifiers. In: NeurIPS, pp 6967–6976

Dabouei, A., Soleymani, S., Taherkhani, F., & Nasrabadi, N.M. (2020). Supermix: Supervising the mixing data augmentation. arXiv:2003.05034

Du, W., Cote, D., & Liu, Y. (2023). Saits: Self-attention-based imputation for time series. *Expert Systems with Applications, 219*(119), Article 619.

Düker, M. C., Matteson, D. S., Tsay, R. S., & Wilms, I. (2025). Vector autoregressive moving average models: A review. *WIREs Computational Statistics, 17*(1), Article e70,009.

Galassi, A., Lippi, M., & Torroni, P. (2020). Attention in natural language processing. *IEEE Transactions on Neural Networks and Learning Systems, 32*(10), 4291–4308.

Gamboa, J.C.B. (2017). Deep learning for time-series analysis. arXiv:1701.01887

Godahewa, R., Bergmeir, C., Webb, G.I., Hyndman, R.J., & Montero-Manso, P. (2021). Monash time series forecasting archive. arXiv preprint arXiv:2105.06643

Gruver, N., Finzi, M., Qiu, S., & Wilson, A.G. (2024). Large language models are zero-shot time series forecasters. Advances in Neural Information Processing Systems 36

Guo, M. H., Xu, T. X., Liu, J. J., Liu, Z.-N., Jiang, P.-T., Mu, T.-J., Zhang, S.-H., Martin, R. R., Cheng, M.-M., & Hu, S.-M. (2022). Attention mechanisms in computer vision: A survey. *Computational Visual Media, 8*(3), 331–368.

Hamilton, J.D. (1994). Time series analysis, vol 2. Princeton New Jersey

Harvey, A. C. (1990). *Forecasting, structural time series models and the Kalman filter*. Cambridge University Press.

Hassan, M. A., Bailek, N., Bouchouicha, K., & Nwokolo, S. C. (2021). Ultra-short-term exogenous forecasting of photovoltaic power production using genetically optimized non-linear auto-regressive recurrent neural networks. *Renewable Energy, 171*, 191–209.

Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation, 9*(8), 1735–1780.

Ismail, A.A., Gunady, M., Bravo, H.C., & Feizi, S. (2020). Benchmarking deep learning interpretability in time series predictions. arXiv:2010.13924

Jin, M., Wang, S., Ma, L., Chu, Z., Zhang, J.Y., Shi, X., Chen, P.-Y. Liang, Y., Li, Y.-F., Pan, S. others (2023). Time-llm: Time series forecasting by reprogramming large language models. arXiv preprint arXiv:2310.01728

Kingma, D.P., & Ba, J. (2014). Adam: A method for stochastic optimization. arXiv:1412.6980
Kolehmainen, M., Martikainen, H., & Ruuskanen, J. (2001). Neural networks and periodic components used in air quality forecasting. *Atmospheric Environment, 35*(5), 815–825.
Lai, G., Chang, W.C., Yang, Y., & Liu, H. (2018). Modeling long and short-term temporal patterns with deep neural networks. In: International Conference on Information Retrieval, pp 95–104
Lea, C., Vidal, R., Reiter, A., & Hager, G.D. (2016). Temporal convolutional networks: A unified approach to action segmentation. In: Computer Vision–ECCV 2016 Workshops: Amsterdam, The Netherlands, October 8-10 and 15-16, 2016, Proceedings, Part III 14, Springer, pp 47–54
LeBaron, B., Arthur, W. B., & Palmer, R. (1999). Time series properties of an artificial stock market. *Journal of Economic Dynamics & Control, 23*(9–10), 1487–1516.
LeCun, Y., Bengio, Y., & Hinton, G. (2015). *Deep learning. nature, 521*(7553), 436–444.
Leung, K.K., Rooke, C., Smith, J., Zuberi, S., & Volkovs, M. (2023). Temporal dependencies in feature importance for time series predictions. ICLR
Liu, A., Feng, B., Xue, B., Wang, B., Wu, B., Lu, C., Zhao, C., Deng, C., Zhang, C., Ruan, C., and others (2024) Deepseek-v3 technical report. arXiv:2412.19437
Lütkepohl, H. (2013). Vector autoregressive models. In: Handbook of research methods and applications in empirical macroeconomics. Edward Elgar Publishing, p 139–164
Makridakis, S., & Hibon, M. (2000). The M3-competition: Results, conclusions and implications. *International Journal of Forecasting, 16*(4), 451–476. https://doi.org/10.1016/S0169-2070(00)00057-1
Makridakis, S., Andersen, A., Carbone, R., Fildes, R., Hibon, M., Lewandowski, R., Newton, J., Parzen, E., & Winkler, R. (1982). The accuracy of extrapolation (time series) methods: Results of a forecasting competition. *Journal of Forecasting, 1*(2), 111–153. https://doi.org/10.1002/for.3980010202
Makridakis, S., Spiliotis, E., & Assimakopoulos, V. (2020). The M4 competition: 100,000 time series and 61 forecasting methods. *International Journal of Forecasting, 36*(1), 54–74.
Makridakis, S., Spiliotis, E., & Assimakopoulos, V. (2022). M5 accuracy competition: Results, findings, and conclusions. *International Journal of Forecasting, 38*(4), 1346–1364. https://doi.org/10.1016/j.ijforecast.2021.11.013
Makridakis, S., Spiliotis, E., Assimakopoulos, V., Chen, Z., Gaba, A., Tsetlin, I., & Winkler, R. L. (2022). The M5 uncertainty competition: Results, findings and conclusions. *International Journal of Forecasting, 38*(4), 1365–1385.
Middlehurst, M., Schäfer, P., & Bagnall, A. (2024). Bake off redux: A review and experimental evaluation of recent time series classification algorithms. *Data Mining and Knowledge Discovery, 38*(4), 1958–2031.
Molnar, C. (2020). Interpretable machine learning. Lulu. com
Nelson, B. K. (1998). Time series analysis using autoregressive integrated moving average (ARIMA) models. *Academic Emergency Medicine, 5*(7), 739–744.
Nussbaumer, H. J., & Nussbaumer, H. J. (1982). *The fast Fourier transform*. Springer.
Oikolab, L. (2025). Weather and climate data api for analysts. https://oikolab.com/
Oreshkin, B.N., Carpov, D., Chapados, N., & Bengio, Y. (2019). N-beats: Neural basis expansion analysis for interpretable time series forecasting.arXiv:1905.10437
Pan, Q., Hu, W., & Chen, N. (2021). Two birds with one stone: Series saliency for accurate and interpretable multivariate time series forecasting. In: IJCAI, pp 2884–2891
Ribeiro, M.T., Singh, S., & Guestrin, C. (2016). Model-agnostic interpretability of machine learning. arXiv:1606.05386
Selvaraju, R.R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., & Batra, D. (2017). Grad-cam: Visual explanations from deep networks via gradient-based localization. In: Proceedings of the IEEE international conference on computer vision, pp 618–626
Sezer, O. B., Gudelek, M. U., & Ozbayoglu, A. M. (2020). Financial time series forecasting with deep learning: A systematic literature review: 2005–2019. *Applied Soft Computing, 90*(106), Article 181.
Shih, S. Y., Sun, F. K., & Hy, L. (2019). Temporal pattern attention for multivariate time series forecasting. *Machine Learning, 108*(8–9), 1421–1441.
Shrikumar, A., Greenside, P., & Kundaje, A. (2017). Learning important features through propagating activation differences. arXiv:1704.02685
Sims, C. A. (1980). Macroeconomics and reality. *Econometrica*. https://doi.org/10.2307/1912017
Suresh, H., Hunt, N., Johnson, A., Celi, L.A., Szolovits, P., & Ghassemi, M. (2017). Clinical intervention prediction and understanding using deep networks. arXiv:1705.08498
Szeliski, R. (2022). Computer vision: algorithms and applications. Springer Nature
Tolstikhin, I. O., Houlsby, N., Kolesnikov, A., Beyer, L., Zhai, X., Unterthiner, T., Yung, J., Steiner, A., Keysers, D., Uszkoreit, J., et al. (2021). Mlp-mixer: An all-mlp architecture for vision. *Advances in neural information processing systems, 34*, 24261–24272.
Tsay, R. S. (2013). *Multivariate time series analysis: With R and financial applications*. Wiley.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need. In: NeurIPS, pp 5998–6008

Wan, R., Mei, S., Wang, J., Liu, M., & Yang, F. (2019). Multivariate temporal convolutional network: A deep neural networks approach for multivariate time series forecasting. *Electronics, 8*(8), Article 876.

Wang, J., Wang, Z., Li, J., & Wu, J. (2018). Multilevel wavelet decomposition network for interpretable time series analysis. In: Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, pp 2437–2446

Wang, S., Wu, H., Shi, X., Hu, T., Luo, H., Ma, L., Zhang, J.Y., & Zhou, J. (2024). Timemixer: Decomposable multiscale mixing for time series forecasting. arXiv:2405.14616

Wei, X., Lei, B., & Ouyang, H. (2020). Wu, Q (2020) Stock index prices prediction via temporal pattern attention and long-short-term memory. *Advances in Multimedia, 1*(8831), 893.

Wen, Q., Sun, L., Yang, F., Song, X., Gao, J., Wang, X., & Xu, H. (2020). Time series data augmentation for deep learning: A survey. arXiv:2002.12478

Zeger, S. L., Thomas, D., Dominici, F., Samet, J. M., Schwartz, J., Dockery, D., & Cohen, A. (2000). Exposure measurement error in time-series studies of air pollution: Concepts and consequences. *Environmental Health Perspectives, 108*(5), 419–426.

Zeiler, M.D., & Fergus, R. (2014). Visualizing and understanding convolutional networks. In: Computer Vision–ECCV 2014: 13th European Conference, Zurich, Switzerland, September 6-12, 2014, Proceedings, Part I 13, Springer, pp 818–833

Zhang, H., Cisse, M., Dauphin, Y.N., & Lopez-Paz, D. (2017a). mixup: Beyond empirical risk minimization. arXiv:1710.09412

Zhang, S., Guo, B., Dong, A., He, J., Xu, Z., & Chen, S. X. (2017b). Cautionary tales on air-quality improvement in Beijing. *Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences, 473*(2205), Article 20170,457.

Zhang, Y., Sun, Q., Qi, D., Liu, J., Ma, R., & Petrosian, O. (2023). Shaptime: A general xai approach for explainable time series forecasting. In: Proceedings of SAI Intelligent Systems Conference, Springer, pp 659–673

Zhang, Y., Tino, P., Leonardis, A., & Tang, K. (2021). A survey on neural network interpretability. *IEEE Transactions on Emerging Topics in Computational Intelligence, 5*(5), 726–742.

Zhou, H., Zhang, S., Peng, J., Zhang, S., Li, J., Xiong, H., & Zhang, W. (2021). Informer: Beyond efficient transformer for long sequence time-series forecasting. In: Proceedings of the AAAI conference on artificial intelligence, pp 11106–11115

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

Authors and Affiliations

Qingyi Pan[1,2] · Liyuan Wang[2] · Xiaoyu Chen[2] · Ning Chen[2]

✉ Ning Chen
ningchen@mail.tsinghua.edu.cn

Qingyi Pan
pqy22@mails.tsinghua.edu.cn

Liyuan Wang
wly19@tsinghua.org.cn

Xiaoyu Chen
chenxiaoyu1025@gmail.com

[1] Department of Statistics and Data Science, Tsinghua University, Beijing, China

[2] Department of Computer Science and Technology, Tsinghua University, Beijing, China